Image Lab：互動式影像處理與人臉分析工具

數位影像處理（DIP）期中作業　技術報告

林奕丞　C113151143

2026-11-09

目　錄

1　專案概述與環境建置.....3

1.1　專案目標.....3

1.2　開發環境.....3

1.3　安裝與執行.....7

1.4　專案結構.....7

2　可執行連結（Live Demo / Source Code）.....8

3　程式功能與操作說明書.....9

3.1　系統架構與資料流.....9

3.2　測試影像與測試方式.....10

3.3　功能 1：影像載入與基礎解析.....11

3.4　功能 2：色彩空間轉換與同步直方圖.....12

3.5　功能 3：自訂卷積核空間濾波與 PSNR.....14

3.6　功能 4：人臉偵測.....23

3.7　PSNR 公式與濾波效果評估.....28

4　AI 協作與完整 Prompt 歷程（AI Audit Trail）.....33

4.1　使用的模型.....33

4.2　協作流程.....33

4.3　AI 的失誤與修正.....33

4.4　完整 Prompt 歷程（依時間序）.....34

5　結論與已知限制.....57

參考資料.....58

1 專案概述與環境建置

1.1 專案目標

本專案以 **Python 3.12 + Streamlit** 開發一個互動式影像處理與人臉分析工具，涵蓋四大功能：

功能	內容
功能 1 影像載入與解析	支援 PNG、JPG/JPEG、BMP；即時預覽並顯示解析度、色彩維度、檔案大小等中繼資訊
功能 2 色彩空間與同步直方圖	Original、RGB 分量、YUV（BT.601）分量三種檢視；切換時直方圖（橫軸 0–255、縱軸像素數）同步更新
功能 3 自訂卷積濾波與 PSNR	使用者輸入奇數方陣核（3×3、5×5…）、可選正規化；濾波前後並排對照；即時計算 PSNR（dB）
功能 4 人臉偵測	OpenCV Haar-Cascade，標記所有人臉邊界框，顯示人臉總數與每張臉的信心度

1.2 開發環境

開發語言版本：Python 3.12（記錄於 `.python-version`；`pyproject.toml` 設定 `requires-python = ">=3.12"`），高於建議的 Python 3.9+。

項目	版本／說明
套件管理	uv（`pyproject.toml` + `uv.lock`）；部署用 `requirements.txt` 由 `uv export --no-dev --no-hashes` 產生
介面	Streamlit 1.67

數值運算	NumPy 2.5（卷積、色彩轉換、直方圖、PSNR 皆以 NumPy 實作）
影像解碼	Pillow 13
人臉偵測	opencv-python-headless 4.14（**必須為 4.x**，5.x 已移除 Haar-Cascade）
繪圖	Matplotlib 3.11
測試與檢查	pytest、ruff（開發用，不列入 `requirements.txt`）
系統套件	`fonts-noto-cjk`（`packages.txt`，雲端部署時安裝中文字型）

第三方相依套件清單（`requirements.txt` 條目說明）：`requirements.txt` 共 41 個固定版本的套件，其中 5 個為**直接相依**（專案程式碼直接使用），其餘 36 個為 Streamlit、Matplotlib 等的**間接相依**（「被誰需要」欄標出來源；`dip-mid` 即本專案）。

套件	版本	類型	被誰需要	用途
`altair`	6.3.0	間接	streamlit	宣告式圖表（Streamlit 圖表元件）
`anyio`	4.16.0	間接	starlette、streamlit	非同步 I/O 抽象層
`attrs`	26.1.0	間接	jsonschema、referencing	類別屬性定義輔助
`certifi`	2026.10.23	間接	requests	HTTPS 憑證根清單
`charset-normalizer`	3.5.2	間接	requests	文字編碼偵測
`click`	8.5.1	間接	streamlit、uvicorn	命令列介面工具
`contourpy`	1.4.0	間接	matplotlib	等高線計算（Matplotlib）
`cycler`	0.12.1	間接	matplotlib	繪圖樣式循環（Matplotlib）
`fonttools`	4.68.0	間接	matplotlib	字型讀取與處理（Matplotlib）
`h11`	0.16.0	間接	uvicorn	HTTP/1.1 協定實作

idna	3.20	間接	anyio、requests	國際化網域名稱處理
itsdangerous	2.2.0	間接	streamlit	資料簽章
jinja2	3.1.6	間接	altair、pydeck	範本引擎
jsonschema	4.26.0	間接	altair	JSON 結構驗證
jsonschema-specifications	2025.9.1	間接	jsonschema	JSON Schema 規格資料
kiwisolver	1.5.1	間接	matplotlib	約束求解（Matplotlib 版面配置）
markupsafe	3.0.4	間接	jinja2	範本字串跳脫
matplotlib	3.11.2	**直接**	本專案	直方圖繪製
narwhals	2.28.0	間接	altair	資料框相容層
numpy	2.5.4	**直接**	本專案、contourpy、matplotlib…	陣列運算：卷積、色彩轉換、直方圖、PSNR
opencv-python-headless	4.14.0.94	**直接**	本專案	Haar-Cascade 人臉偵測與邊界框繪製（無 GUI 版本，適合雲端）
packaging	26.3	間接	altair、matplotlib、streamlit	版本字串解析
pandas	3.1.1	間接	streamlit	表格資料處理（Streamlit 表格顯示）
pillow	13.0.0	**直接**	本專案、matplotlib、streamlit	影像解碼（PNG／JPG／BMP）、EXIF 方向處理
protobuf	7.36.2	間接	streamlit	序列化格式（Streamlit 前後端通訊）

pyarrow	26.0.1	間接	streamlit	欄式資料格式（Streamlit 資料傳輸）
pydeck	0.9.3	間接	streamlit	地圖視覺化（Streamlit）
pyparsing	3.3.3	間接	matplotlib	文字剖析（Matplotlib）
python-dateutil	4.0.0	間接	matplotlib、pandas	日期時間處理
python-multipart	0.0.33	間接	streamlit	表單與檔案上傳解析
referencing	0.37.0	間接	jsonschema、jsonschema-specifications	JSON 參照解析
requests	2.34.2	間接	streamlit	HTTP 用戶端
rpds-py	2026.9.1	間接	jsonschema、referencing	持久性資料結構
starlette	1.10.0	間接	streamlit	ASGI 網頁框架（Streamlit 伺服器）
streamlit	1.67.0	**直接**	本專案	互動式網頁介面（本專案的 UI 框架）
typing-extensions	4.16.0	間接	altair、anyio、referencing…	型別提示向後相容
tzdata	2026.6	間接	pandas、python-dateutil	時區資料
urllib3	2.8.0	間接	requests	HTTP 連線函式庫
uvicorn	0.54.0	間接	streamlit	ASGI 伺服器
watchdog	7.1.0	間接	streamlit	檔案變更監看
websockets	17.4	間	streamlit	WebSocket 實作

		接		

1.3 安裝與執行

```
git clone https://github.com/koraqq/DIP-mid
cd DIP-mid
uv sync
uv run streamlit run app.py
```

不使用 uv 時：建立虛擬環境後執行 `pip install -r requirements.txt`，再執行 `streamlit run app.py`。部署所需的系統套件（中文字型 `fonts-noto-cjk`）列於 `packages.txt`。

1.4 專案結構

```
DIP-mid/
├── app.py                Streamlit 進入點（四個分頁）
├── imagelab/
│   ├── io/               loader.py、metadata.py
│   ├── color/            channels.py、yuv.py
│   ├── analysis/         histogram.py
│   ├── filtering/        kernels.py、convolution.py、metrics.py
│   ├── validation/       matrix_parser.py、guards.py
│   ├── detection/        haar_detector.py、draw.py、base.py
│   └── ui/               sidebar、tab_overview、tab_color、tab_filter、tab_face、state
├── tests/                pytest 單元測試（含邊界條件）
├── docs/                 psnr.md、convolution.md、report/（本報告）、images/
├── prompt_log/           AI 協作完整紀錄
├── requirements.txt、packages.txt、pyproject.toml、uv.lock
```

2　可執行連結（Live Demo / Source Code）

本專案以 Streamlit Community Cloud 作為線上即時展示，不另提供 Colab。兩個連結的檢閱權限皆為 **Public**（不需登入即可開啟）。

項目	連結
線上即時展示（Streamlit Cloud，Public）	https://dip-mid-tb8vrujcpzjqkwkc5onscz.streamlit.app/
版本控制儲存庫與完整原始碼（GitHub，Public）	https://github.com/koraqq/DIP-mid
完整對話紀錄（AI Audit Trail 原文）	https://claude.ai/share/c7f80221-4bbc-4509-ade9-f5ee6943d3d8

3　程式功能與操作說明書

本章依序說明系統架構與資料流（3.1）、測試方式（3.2）、四項功能的操作與測試案例分析（3.3–3.6），以及 PSNR 的公式定義與濾波效果的客觀評估（3.7）。

3.1　系統架構與資料流

3.1.1　分層架構

專案採「**介面層與演算法層分離**」：`imagelab/ui/` 只負責呈現與互動，所有影像處理邏輯放在不依賴 Streamlit 的演算法層，因此每個模組都能單獨以 pytest 測試，也能在 Colab 等其他環境重複使用。

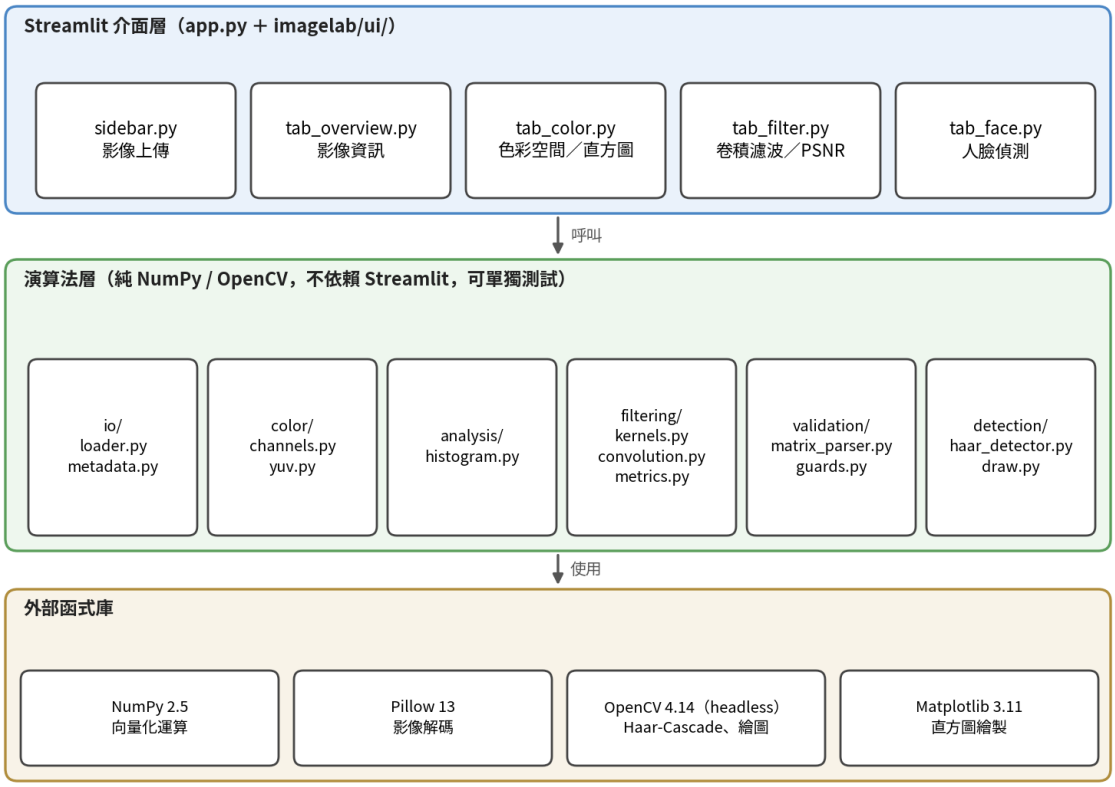


圖 1　系統分層架構圖

3.1.2　資料流

使用者上傳的檔案先由 `load_image()` 統一解碼為 **RGB、uint8、(H, W, 3)** 的 NumPy 陣列，之後四個分頁共用這份資料。解碼結果以 `st.cache_data` 快取，切換分頁或調整參數時不需重新解碼；濾波與人臉偵測也各有快取，參數不變就不重算。

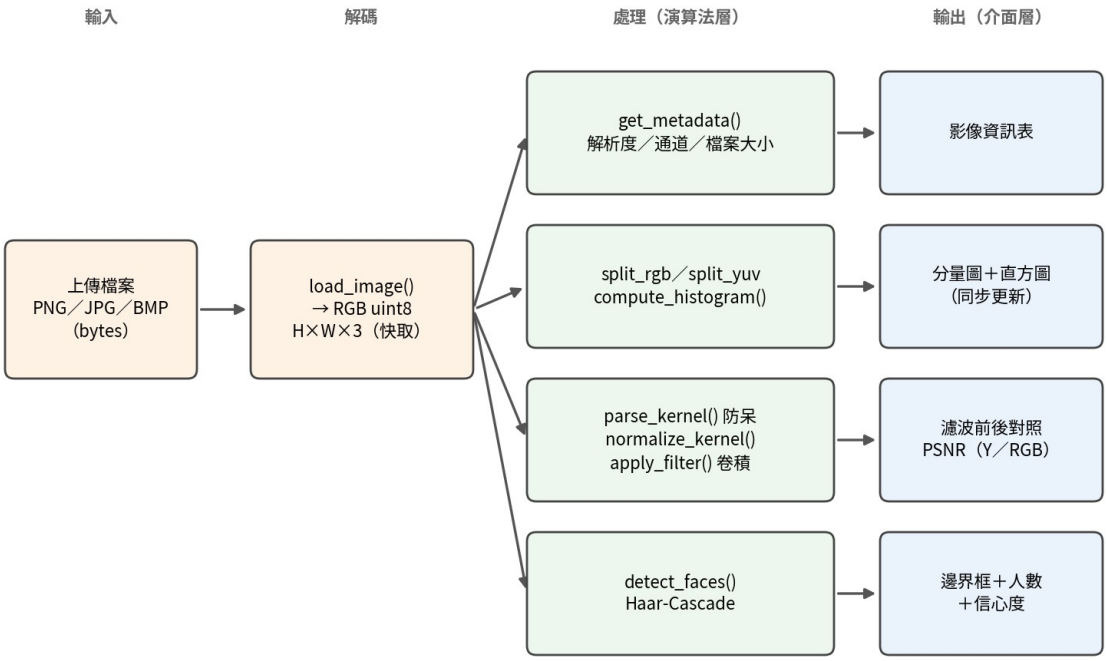


圖 2　資料流圖

3.1.3　模組職責

模組	職責	設計重點
io	影像解碼與中繼資訊	依檔案**內容**（非副檔名）判斷格式；套用 EXIF 方向；透明背景合成白底；拒絕過大影像（> 2500 萬像素）
color	RGB 分離、YUV 轉換	BT.601 全範圍；轉換矩陣寫於檔案註解
analysis	直方圖計算與繪圖	橫軸 0–255、縱軸像素數；自動偵測中文字型，偵測不到時標題改英文
filtering	核、卷積、PSNR	卷積以 NumPy 向量化實作；先轉 float64 再相減，避免 uint8 溢位
validation	輸入防呆	指出出錯的列與格；提供可顯示的中文錯誤訊息
detection	Haar 偵測與繪圖	超大圖先縮小再偵測；對分類器加鎖以支援多人同時連線

3.2　測試影像與測試方式

測試影像為一張 800×533 的 JPEG 照片（含一張清晰人臉與兩張失焦人臉）。本影像僅供本課程作業之非營利教學與展示用途，不作任何商業使用；作業繳交後儲存庫將改為私人。

3.3　功能 1：影像載入與基礎解析

上傳後即時預覽，並以表格顯示檔名、格式、解析度、像素總數、原始色彩模式、通道數與檔案大小（圖 3：本例為 JPEG、800×533、426,400 像素、RGB 三通道、46.52 KB）。



圖 3　功能 1：影像預覽與中繼資訊

測試案例（邊界條件）

測試案例	預期行為	對應單元測試
PNG、BMP 編碼後再解碼	與原陣列逐元素相同（無損）	`test_lossless_formats_roundtrip`
JPEG	以 (H, W, 3) 的 RGB 載入	`test_jpeg_loads_as_rgb`
灰階 PNG	自動擴為三通道，三個通道值相同	`test_grayscale_png_becomes_three_channels`
含透明通道的 PNG	合成在白色背景上（全透明 → 全白）	`test_rgba_composited_on_white`
空檔案、內容損毀	顯示友善錯誤訊息，程式不當機	`test_empty_and_garbage_raise_friendly_error`
不支援的格式（GIF）	提示不支援並說明可用格式	`test_unsupported_format_rejected`

超過 2500 萬像素	提示影像過大	test_too_large_rejected

3.4　功能 2：色彩空間轉換與同步直方圖

RGB 通道分離：直接取 R、G、B 三個 2D 陣列，可選「著色」（只保留該通道、其餘為 0）或「灰階」顯示。

YUV 轉換（BT.601 全範圍，U = Cb、V = Cr）：

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.168736 & -0.331264 & 0.5 \\ 0.5 & -0.418688 & -0.081312 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix}$$

反轉換為 $R = Y + 1.402(V - 128)$、$G = Y - 0.344136(U - 128) - 0.714136(V - 128)$、$B = Y + 1.772(U - 128)$。

同步直方圖：每個顯示模式下，影像與其對應通道的直方圖在同一次繪製中由同一份資料計算：Original 顯示灰階（亮度）與 RGB 疊圖直方圖；RGB 模式顯示 R、G、B 各自的影像與直方圖；YUV 模式顯示 Y、U、V 各自的影像與直方圖。因此切換模式時，影像與直方圖不可能不同步。

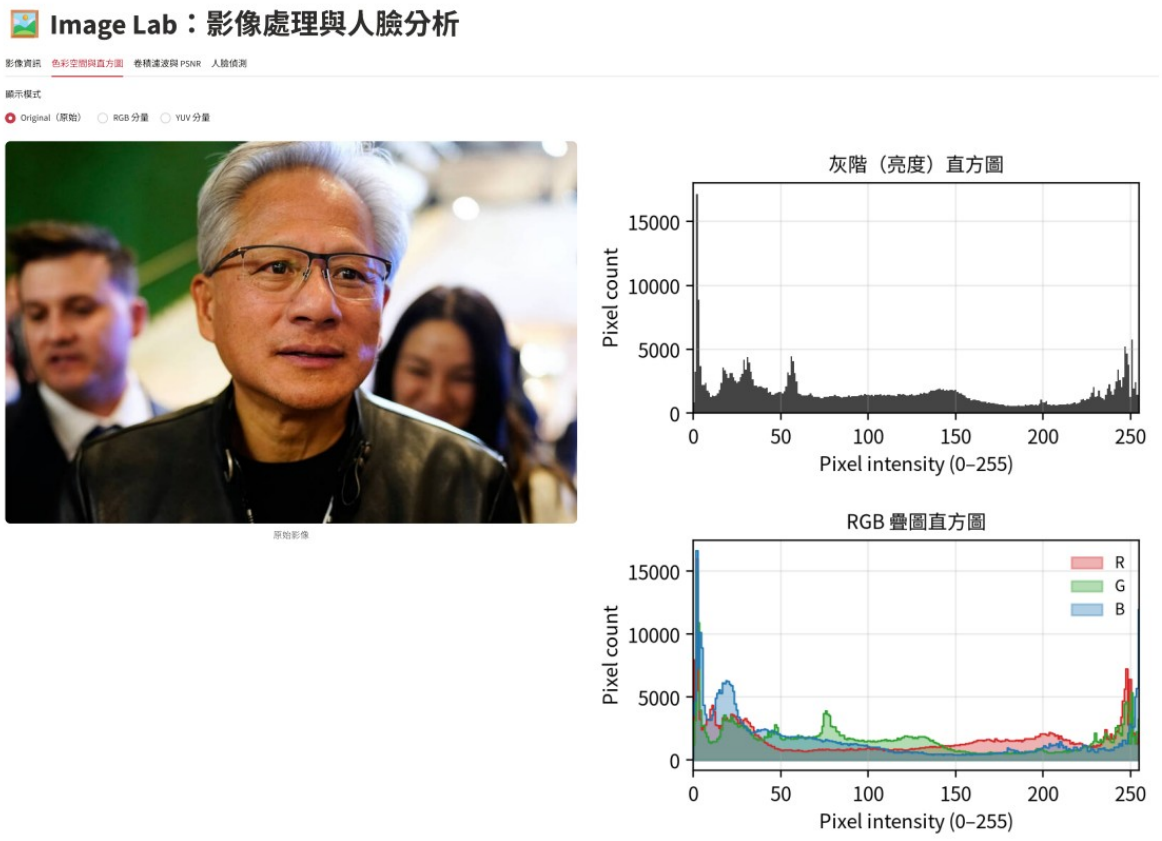


圖 4　Original 模式：灰階直方圖與 RGB 疊圖直方圖

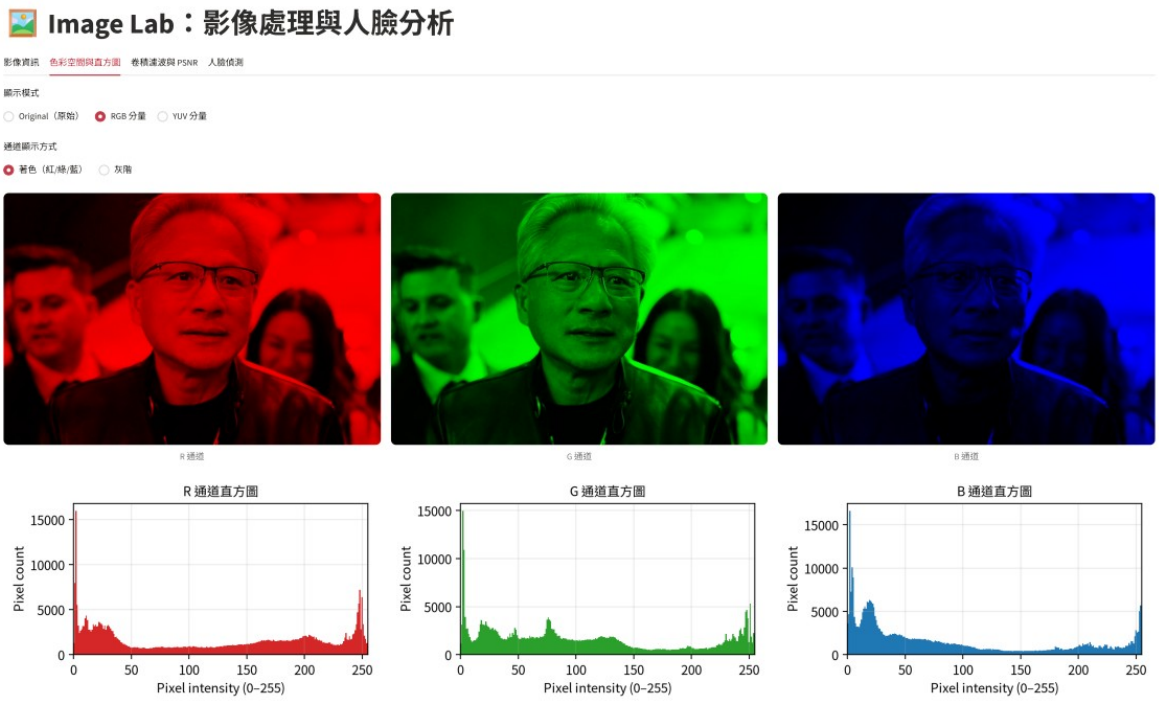


圖 5　RGB 分量模式：三個通道圖與各自的直方圖

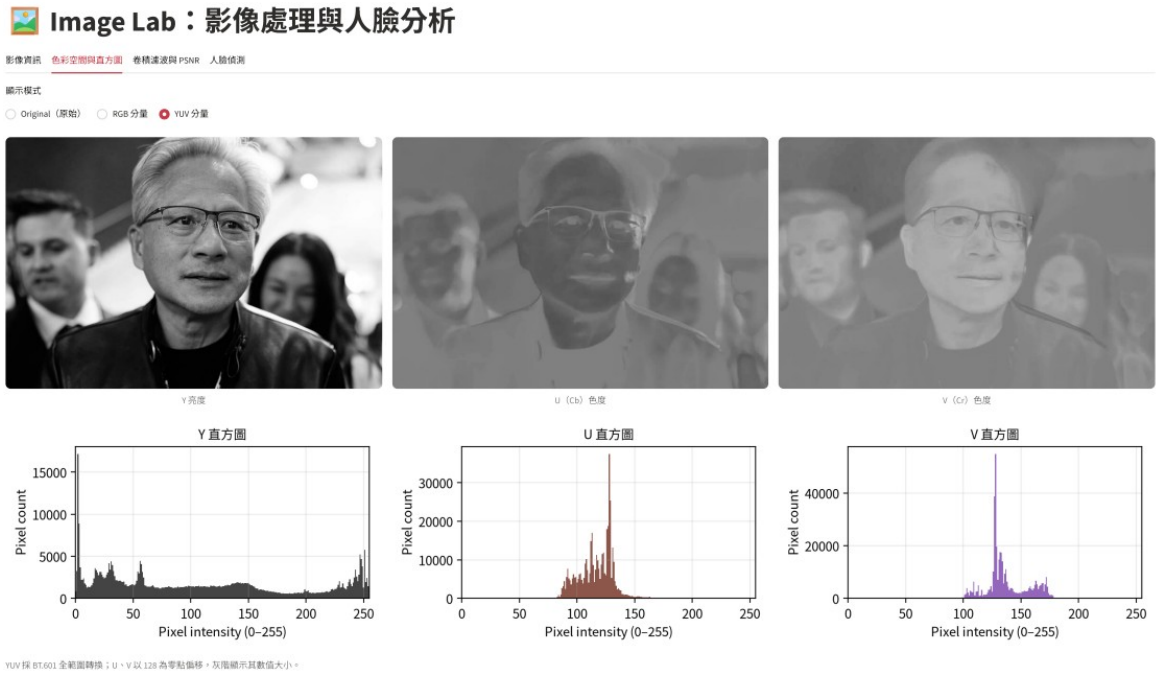


圖 6　YUV 分量模式：Y、U、V 分量圖與各自的直方圖

觀察：本例的 U、V 直方圖都集中在 128 附近、範圍很窄，所以 U、V 分量圖看起來偏灰、對比低；這是因為多數像素色彩飽和度不高，色度偏離中性（128）的程度小。Y 分量則涵蓋整個亮度範圍，與灰階影像相同。

測試案例

測試案例	預期行為	對應單元測試
灰階像素 (R=G=B=90)	Y = 90、U = V = 128（無色度）	test_gray_pixels_have_neutral_chroma
純紅 (255, 0, 0)	(Y, U, V) = (76, 85, 255)	test_known_primaries
RGB → YUV → RGB 往返	每個通道誤差 ≤ 3	test_roundtrip_error_small
直方圖總和	長度 256，總和等於像素數	test_histogram_length_and_total
非 uint8 輸入	拒絕並報錯	test_rejects_non_uint8
系統沒有中文字型	圖表標題退回英文，不出現方框	test_title_falls_back_to_english_without_cjk_font

3.5 功能 3：自訂卷積核空間濾波與 PSNR

3.5.1 卷積定義與實作

核 K 為 $k \times k$（k 為奇數），中心偏移 $c = \lfloor k/2 \rfloor$，**卷積**定義為：

$$out[i,j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} K[m,n]\, x[i-m+c, j-n+c]$$

令補邊後的影像為 P（四邊各補 c 格，$P[a,b] = x[a-c, b-c]$），並令 $u = k-1-m$、$v = k-1-n$，則 $x[i-m+c, j-n+c] = P[i+u, j+v]$，因此

$$out[i,j] = \sum_{u} \sum_{v} K'[u,v]\, P[i+u, j+v], \quad K'[u,v] = K[k-1-u, k-1-v]$$

也就是「**把核旋轉 180° 後，在補邊影像上做滑動視窗內積**」。程式只迴圈 k^2 次，每次是整張影像的向量運算（`out += K'[u,v] * P[u:u+H, v:v+W]`），不逐像素迴圈；本專案**不使用** `cv2.filter2D`，僅在測試中拿它對照結果。對稱核（高斯、均值、拉普拉斯）卷積與相關結果相同；不對稱核（Sobel、浮雕）是否翻轉會改變方向，所以測試特別用不對稱核驗證。

邊界處理可選：補零（zero）、鏡射（reflect，不重複邊緣像素）、複製邊緣（replicate）。

正規化：除以核元素總和 $s = \sum K$，使平均亮度不變（對亮度為 a 的常數影像，輸出為 $a \sum K / s = a$）。總和為 0 的核（Sobel、Laplacian）無法相除，程式會**略過正規化並提示**，避免除以零。

輸出轉換：卷積結果為浮點數，加上偏移後四捨五入並截斷到 $[0, 255]$，轉回 uint8。對含負響應的核（邊緣類），可勾選「加 128 偏移」讓正負響應都能顯示。

3.5.2　預設範本

範本	核	正規化	偏移
Identity	$[0,0,0;0,1,0;0,0,0]$	否	0
Box 均值模糊 3×3	全 1	是（÷9）	0
Gaussian 3×3	$[1,2,1;2,4,2;1,2,1]$	是（÷16）	0
Gaussian 5×5	二項式係數 $[1,4,6,4,1]$ 外積	是（÷256）	0
Sharpen 銳化	$[0,-1,0;-1,5,-1;0,-1,$	否（總和 1）	0
Emboss 浮雕	$[-2,-1,0;-1,1,1;0,1,2]$	否（總和 1）	0
Sobel X	$[-1,0,1;-2,0,2;-1,0,1]$	略過（總和 0）	128
Sobel Y	$[-1,-2,-1;0,0,0;1,2,1]$	略過（總和 0）	128
Laplacian	$[0,1,0;1,-4,1;0,1,0]$	略過（總和 0）	128

3.5.3　實測畫面

每張畫面左側為核輸入與設定，右側由左至右為濾波前、濾波後、差異圖（×4 放大），下方為 PSNR。各範本的 PSNR 數值彙整於第 3.7 節。



圖 7　Box 均值模糊 3×3



圖 8　Gaussian 3×3



圖 9　Gaussian 5×5



圖 10　Sharpen 銳化

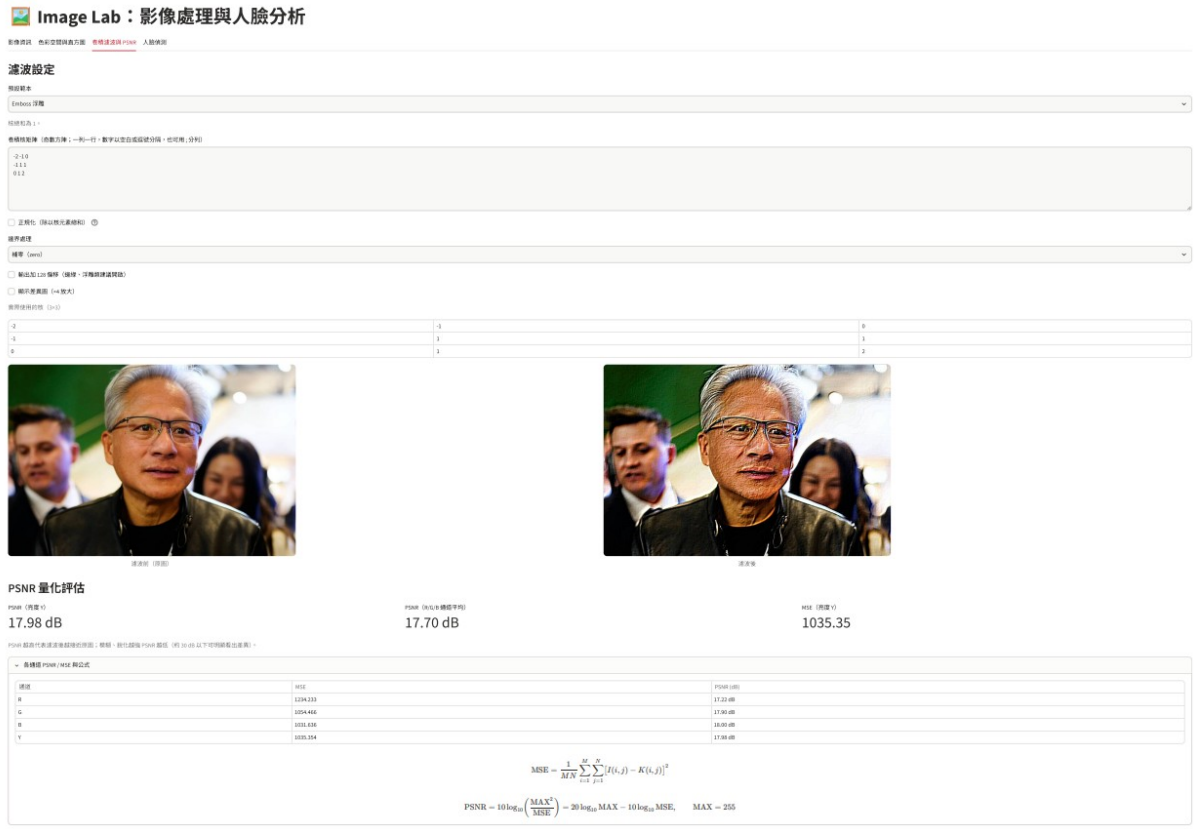


圖 11　Emboss 浮雕

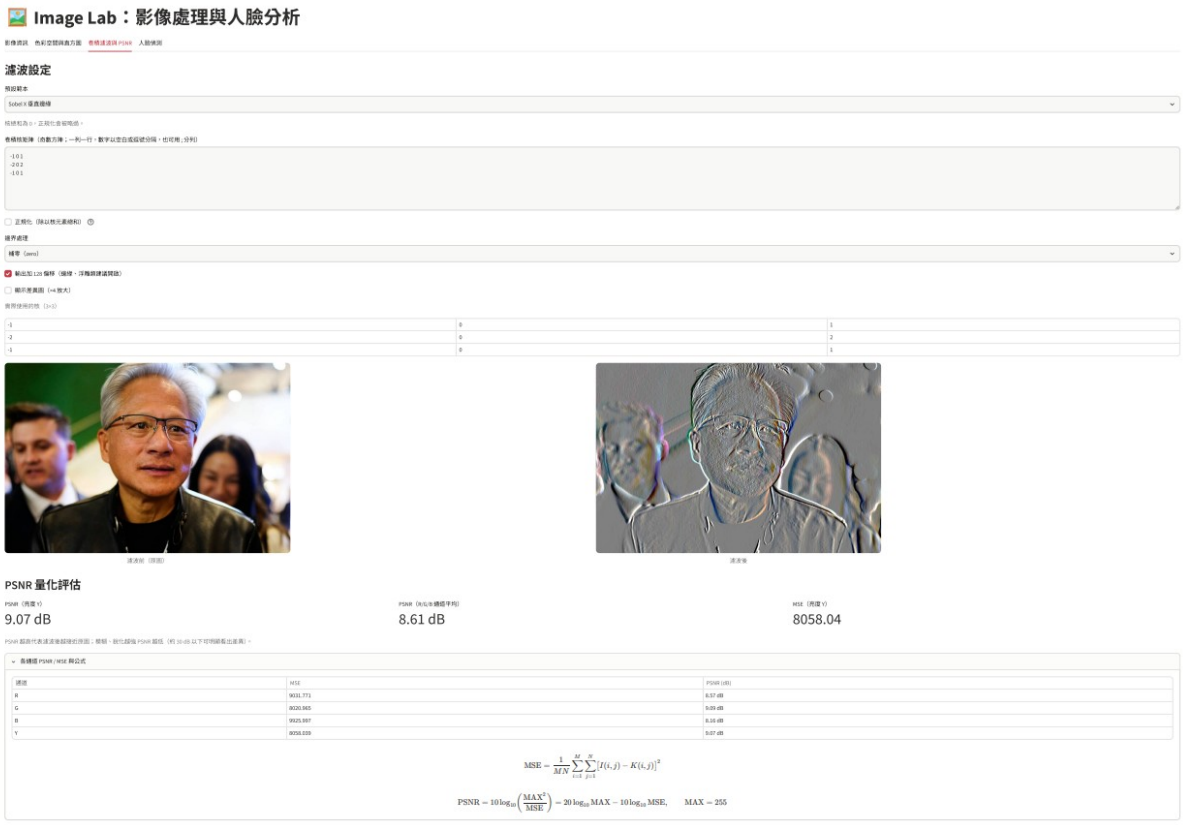


圖 12　Sobel X（垂直邊緣）

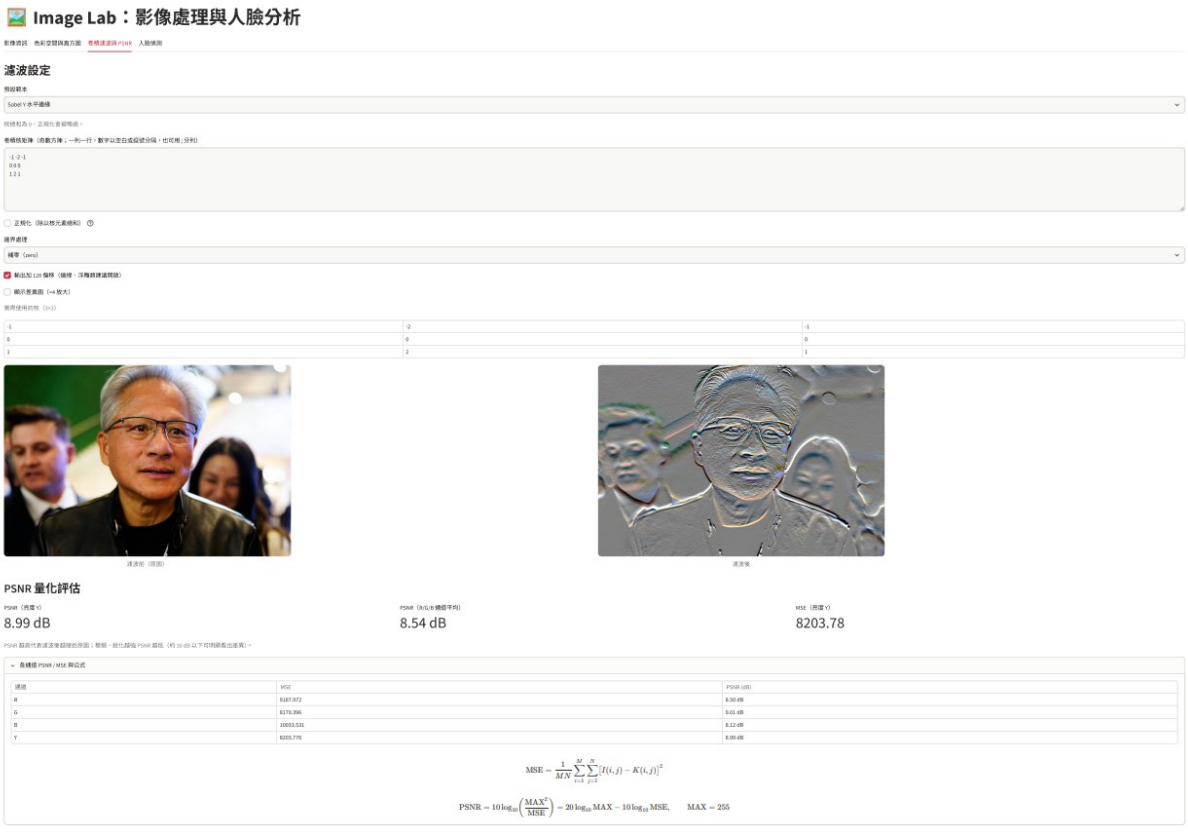


圖 13　Sobel Y（水平邊緣）

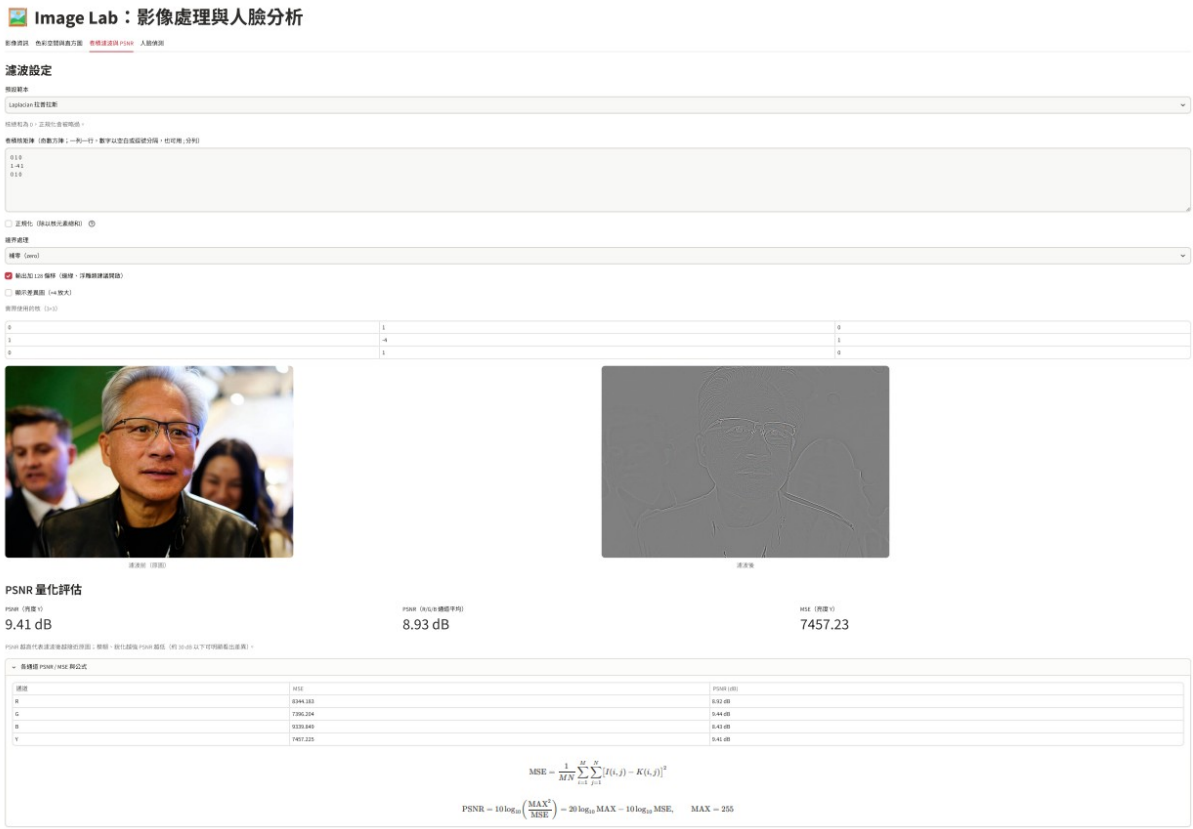


圖 14　Laplacian

3.5.4　邊界條件與輸入防呆

測試案例	預期行為	驗證方式
非方陣（如 3×4）	顯示「必須是方陣」，不執行運算	test_non_square_rejected、圖 15
偶數尺寸（如 2×2、4×4）	顯示「必須是奇數」（偶數沒有中心點）	test_even_size_rejected、圖 16
各列數字個數不同	指出第 1 列與出錯列的個數	test_ragged_rows_rejected
含非數字（如 abc）	指出第幾列第幾格	test_bad_token_reports_position、圖 17
NaN、inf、1e999	拒絕：不是有限數值	test_nan_and_inf_rejected
空白輸入	提示請輸入核矩陣	test_empty_input
超過 15×15 的核	拒絕：核過大	test_too_large_rejected
核比影像還大	拒絕並說明	test_kernel_larger_than_image_rejected
全零核	警告：結果會是全黑	test_warnings
總和為 0 且勾選正規化	略過正規化並提示（不除以零）	test_normalize_skips_zero_sum_kernel、圖 18

原圖與濾波後完全相同（Identity）	MSE = 0，PSNR 顯示 ∞	`test_identical_images_give_infinite_psnr`、圖 19
全形逗號、Unicode 負號、；分列	正常解析	`test_parse_fullwidth_characters` 等
與教科書定義逐元素對照	三種邊界、3×3 與 5×5 非對稱核皆一致	`test_matches_naive_definition_with_asymmetric_kernels`
與 OpenCV `filter2D` 對照	翻轉核後三種邊界皆一致（誤差 < 1e-9）	`test_matches_opencv_filter2d`
輸出截斷與偏移	超過 255 截為 255、負值截為 0；偏移 128 正確	`test_clipping_and_offset`

防呆介面實測：以下 5 張為實際輸入錯誤或特殊核時的畫面。輸入有誤時，左側顯示明確的錯誤或警告訊息，右側提示先修正，程式不會當機。



圖 15　防呆 1：輸入 2×3 的非方陣，顯示「卷積核必須是方陣，目前是 2×3」



圖 16　防呆 2：輸入 2×2 的偶數尺寸核，顯示「尺寸必須是奇數」



圖 17　防呆 3：第 2 列第 3 格輸入 abc，指出是哪一列哪一格不是有效數字

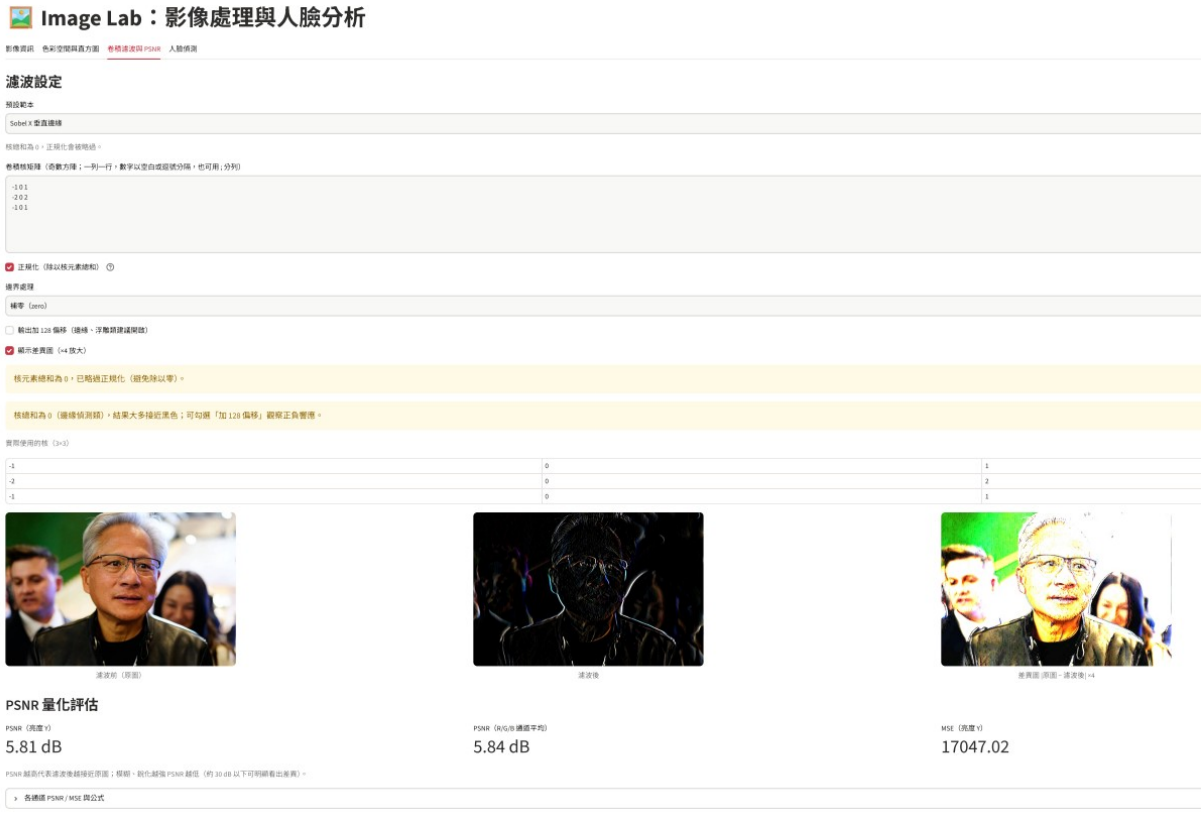


圖 18　防呆 4：Sobel X 勾選正規化，核總和為 0，自動略過並提示（避免除以零）



圖 19　防呆 5：Identity 範本，濾波前後完全相同（MSE = 0），PSNR 顯示 ∞

3.6 功能 4：人臉偵測

3.6.1 演算法

採用 Viola–Jones 的 **Haar-Cascade**（OpenCV 內建的 `haarcascade_frontalface_default.xml`）：以 Haar-like 矩形特徵描述明暗對比；用**積分影像**使任一矩形和只需 4 次查表；以 AdaBoost 挑選特徵組成強分類器；再把多個強分類器串成**級聯（cascade）**，前面的簡單階段能快速排除大多數非人臉區域。偵測時在影像金字塔的多個尺度上以滑動視窗搜尋。

可調參數：`scaleFactor`（金字塔縮放比，越接近 1 越細也越慢）、`minNeighbors`（需要的鄰近候選框數，越大越嚴格）、最小人臉尺寸。

信心度：`detectMultiScale3(..., outputRejectLevels=True)` 會回傳每個框最後一層的**層級權重** w，它沒有上限、也不是機率。本專案以單調函數壓到 0–1：

$$confidence = 1 - e^{-w/2}$$

並同時顯示原始權重。這是**啟發式正規化**，適合比較同一張影像中各框的相對可信度，不代表真實機率。

3.6.2 實測畫面

預設參數（scaleFactor = 1.10、minNeighbors = 5、最小人臉尺寸 = 30）：測試影像中的三張臉都被偵測到，其中包含兩張失焦的臉。

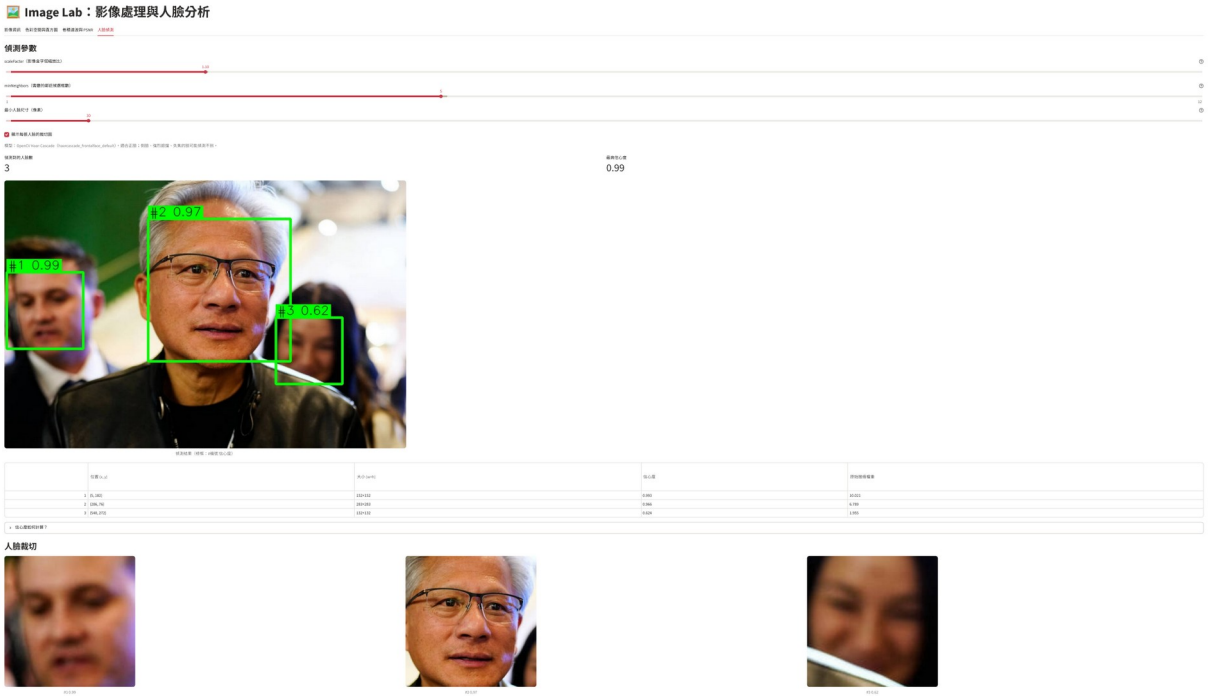


圖 20　預設參數的偵測結果（綠框標籤為 #編號 與信心度）

#	位置 (x, y)	大小 (w×h)	信心度	原始層級權重 w
1	(5, 282)	152×152	0.993	10.02
2	(286, 76)	283×283	0.966	6.79
3	(540, 272)	132×132	0.624	1.96

信心度與原始權重符合 $1-e^{-w/2}$（例如 $w=10.02$ 時為 $1-e^{-5.01}=0.993$）；表中權重取到小數第 2 位，信心度由程式以完整權重計算。三個框依信心度由高到低排序。位置最靠左的失焦人臉（#1）權重反而最高；可能的原因是 Haar 特徵看的是眼、鼻、臉頰之間的粗略明暗結構，輕微失焦仍保有這些粗結構。這只是對單一影像的觀察，並非一般性結論。

參數調整與無人臉的情況

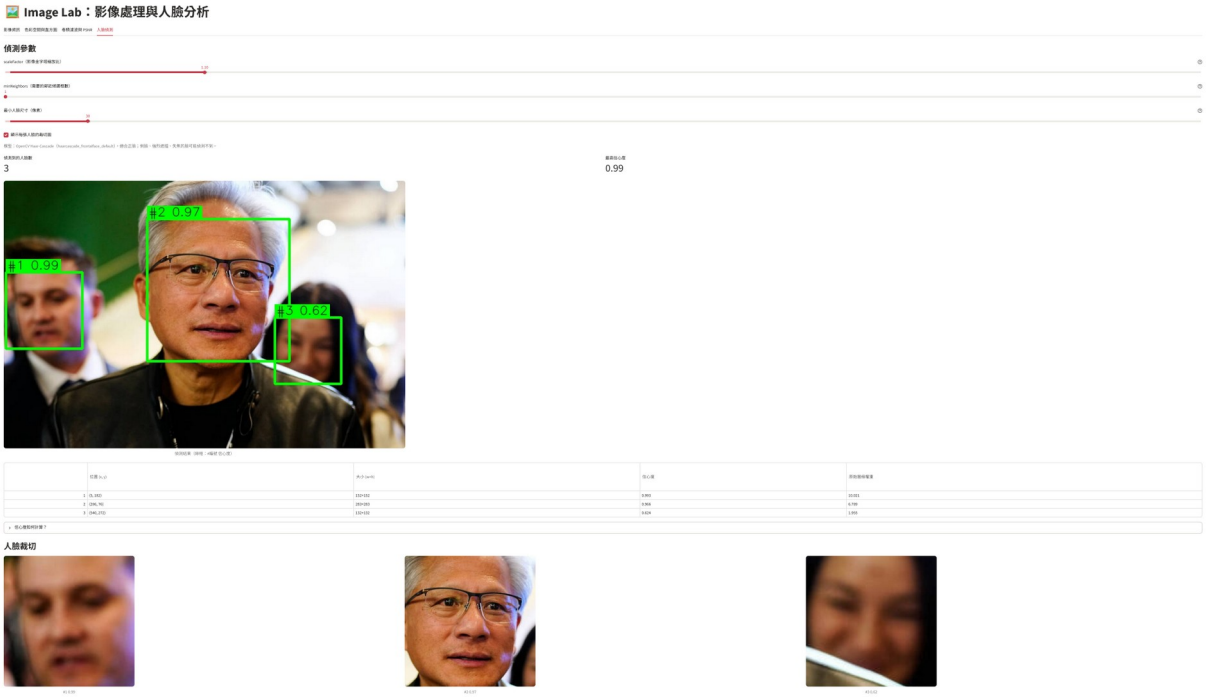


圖 21　minNeighbors = 1（其餘為預設）：仍為 3 張臉，位置與信心度和預設完全相同

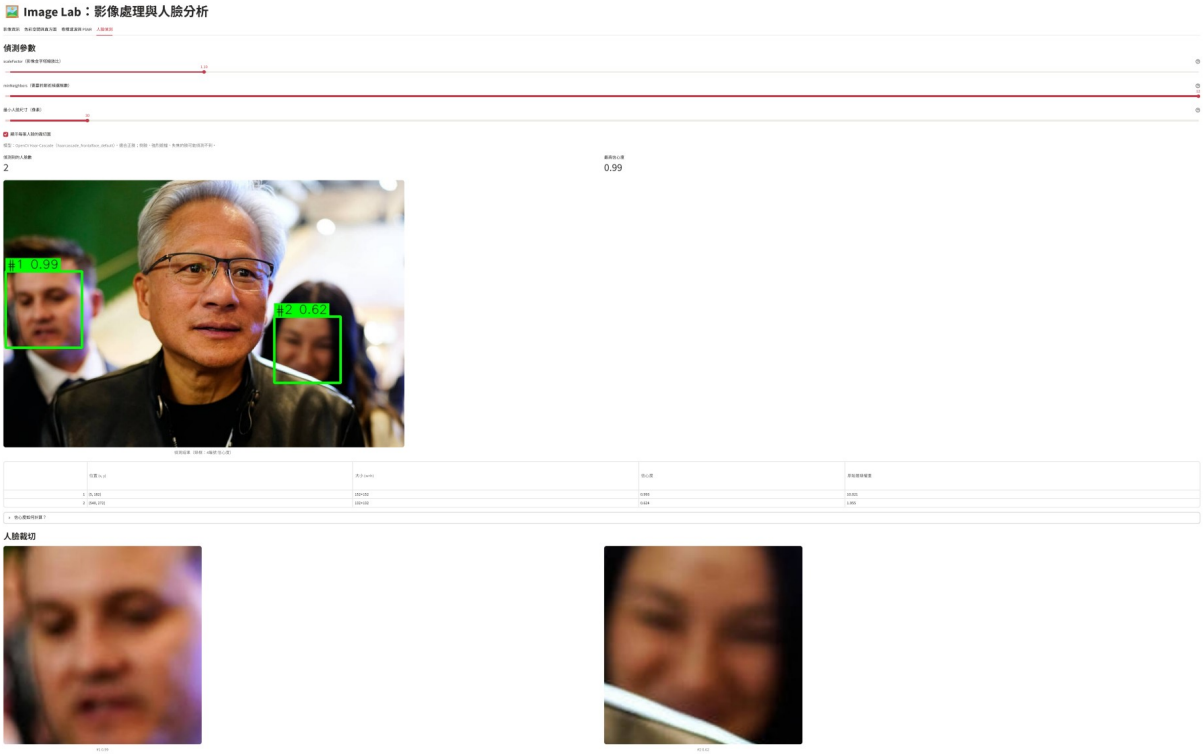


圖 22　minNeighbors = 12（其餘為預設）：只剩 2 張臉，中間較大的清楚人臉不再被保留



圖 23　最小人臉尺寸調大（約 300 像素）：沒有人臉大於此尺寸，偵測 0 張



圖 24　影像中沒有人臉：偵測 0 張，程式不當機

觀察：`minNeighbors` 從預設的 5 降到 1（圖 21），結果仍是同樣的 3 張臉，沒有多出誤判框，表示本例中沒有明顯的假陽性候選；調高到 12（圖 22），保留下來的只有左側失焦的人臉（#1）與右側的臉（原 #3），中間較大、最清楚的人臉被濾掉。

minNeighbors 要求一個候選區域周圍有足夠多個重疊的偵測窗才保留。調高它會讓門檻變嚴格，但嚴格不等於「只留下最可信的」：本例中被濾掉的是信心度 0.966 的大臉，保留的反而有信心度 0.624 的臉。一個可能的解釋是，大尺寸的臉在影像金字塔上能產生的重疊窗較少，所以較不容易達到高門檻（此為推測，未另做實驗驗證）。因此 minNeighbors 與信心度衡量的是不同的東西，調參時兩者需要一起觀察。

3.6.3 邊界條件

測試案例	預期行為	驗證方式
全灰（無任何結構）的影像	偵測 0 張	test_flat_image_has_no_faces
比偵測視窗還小的影像（10×10、1×1）	回傳空清單，不當機	test_tiny_image_returns_empty_without_crashing
最小人臉尺寸大於影像	回傳空清單	test_min_size_larger_than_image_returns_empty
參數不合法（scaleFactor ≤ 1、minNeighbors < 1、非 RGB uint8）	拋出明確錯誤	test_parameter_validation
信心度範圍與單調性	落在 [0, 1) 且隨權重遞增	test_confidence_is_monotonic_and_bounded
畫框	不修改輸入影像；框貼近邊緣時標籤不超出影像	test_draw_*
超大影像（> 1280 像素）	先縮小再偵測，框換算回原圖座標	實作於 haar_detector.py
多人同時連線	對分類器加鎖，避免非執行緒安全造成當機	實作於 haar_detector.py
多人臉、含失焦的臉（本例三張臉，其中兩張失焦）	三張皆偵測到，信心度 0.993、0.966、0.620	圖 20
minNeighbors 調到最低（1）	本例仍為 3 張，沒有多出誤判框	圖 21
minNeighbors 調到最高（12）	本例只剩 2 張，大臉被濾掉	圖 22
最小人臉尺寸大於影像中所有人臉	偵測 0 張，不當機	圖 23
影像中沒有人臉	偵測 0 張，不當機	圖 24

3.6.4 單元測試

所有模組皆以 pytest 測試（tests/）：在開發機（Windows、Python 3.12）上的結果為 **71 個通過、1 個略過**（0 失敗）。略過的是選用的真實影像測試：assets/samples/ 內沒有 face*.jpg 範例圖時會自動略過；其餘包含需要 Haar 模型的偵測測試皆通過。

3.7 PSNR 公式與濾波效果評估

3.7.1 公式推導

設原圖為 I、濾波後影像為 K，尺寸 $M \times N$，8-bit 影像的最大像素值 $MAX = 2^8 - 1 = 255$。

步驟 1：均方誤差（MSE）——每個像素差的平方取平均：

$$MSE = \frac{1}{MN} \sum_{i=1}^{M} \sum_{j=1}^{N} [I(i,j) - K(i,j)]^2$$

步驟 2：訊號功率與雜訊功率之比——峰值訊號功率為 MAX^2，雜訊功率為 MSE，比值為 MAX^2 / MSE。

步驟 3：換成分貝——功率比取 $10 \log_{10}$：

$$PSNR = 10 \log_{10} \left(\frac{MAX^2}{MSE} \right) = 20 \log_{10} MAX - 10 \log_{10} MSE \approx 48.13 - 10 \log_{10} MSE \, (\text{dB})$$

數值驗算：$MSE = 1 \Rightarrow 48.13$ dB；$MSE = 100 \Rightarrow 10 \log_{10}(65025/100) \approx 28.13$ dB；黑對白（$MSE = 65025$）$\Rightarrow 0$ dB；兩圖相同（$MSE = 0$）$\Rightarrow \infty$。MSE 每增為 2 倍，PSNR 減少 $10 \log_{10} 2 \approx 3.01$ dB。

幾何意義：把影像看成 $\mathbb{R}^{MN}$ 中的向量，$MSE = \| x_I - x_K \|_2^2 / (MN)$，亦即兩影像在像素空間中歐氏距離平方的平均；PSNR 是這個距離經 $-10 \log_{10}(\cdot)$ 轉換的結果，距離越近 PSNR 越高。

3.7.2 本專案的計算方式

- **逐通道**：對 R、G、B 分別計算 MSE 與 PSNR。
- **R/G/B 通道平均**：三個通道 PSNR（dB）的算術平均。
- **亮度 Y**：兩張影像先以 BT.601 轉成 Y（四捨五入為 uint8）再計算，較貼近人眼對明暗的感知。
- 先轉 float64 再相減，避免 uint8 溢位（例如 $0 - 255$ 在 uint8 會變成 1）；test_no_uint8_wraparound 驗證了這點。

- $MSE = 0$ 時定義 $PSNR = \infty$，介面顯示「∞」。

3.7.3 各濾波器的實測結果

下表取自第 3.5.3 節各張畫面（同一張測試影像）。Y 的 PSNR 與 MSE 可互相換算驗證（例如 Box：$10\log_{10}(65025/21.80) = 34.75$ dB）。

濾波器	PSNR（亮度 Y）	PSNR（R/G/B 平均）	MSE（亮度 Y）	效果
Box 3×3	34.75 dB	34.56 dB	21.80	均勻模糊
Gaussian 3×3	37.10 dB	36.91 dB	12.69	輕度模糊
Gaussian 5×5	34.50 dB	34.31 dB	23.05	較強模糊
Sharpen	28.65 dB	28.30 dB	88.72	邊緣增強
Emboss	17.98 dB	17.70 dB	1035.35	浮雕、凹凸感
Sobel X	9.07 dB	8.61 dB	8058.04	垂直邊緣圖
Sobel Y	8.99 dB	8.54 dB	8203.78	水平邊緣圖
Laplacian	9.41 dB	8.93 dB	7457.23	二階邊緣圖

截圖設定：本節各張截圖顯示的邊界處理為「補零（zero）」（程式的預設值為鏡射）；正規化與偏移為各預設範本的預設值（Box、Gaussian 勾選正規化；Sobel、Laplacian 勾選「加 128 偏移」；Sharpen、Emboss 兩者皆無）。

邊界處理對結果的影響見第 3.7.4 節。

3.7.4 邊界處理對 PSNR 的影響

為了說明為何本報告的 Gaussian 3×3 為 37.10 dB（補零），而以鏡射測得的結果更高，以同一張影像、同一個 Gaussian 3×3 核（皆勾選正規化），只改變邊界處理：

邊界處理	PSNR（亮度 Y）	PSNR（R/G/B 平均）	MSE（亮度 Y）
補零（zero）	37.10 dB	36.91 dB	12.69
鏡射（reflect）	40.63 dB	40.24 dB	5.62
複製邊緣（replicate）	40.65 dB	40.26 dB	5.59



圖 25　Gaussian 3×3，邊界處理＝補零



圖 26　Gaussian 3×3，邊界處理＝鏡射



圖 27　Gaussian 3×3，邊界處理＝複製邊緣

代數推導：補零時，視窗滑到影像最外圈會有一排格子落在影像外，被當成 0。3×3 高斯核正規化後的權重為 $\frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix}$，最外圈（非角落）像素有一整排（權重 $1+2+1=4$，即 $4/16=25\%$）落在影像外，若該處亮度為 Y，輸出約為 $0.75Y$，誤差約 $0.25Y$。

外圈像素占全部像素的比例為 $\frac{2\times 800+2\times 533-4}{800\times 533}\approx 0.62\%$。若以 $E[Y^2]\approx 17047$ 估算（此值取自 Sobel X 未加偏移時的 MSE：輸出近似 0，所以 $MSE\approx E[Y^2]$），補零額外增加的 MSE 約為

$$0.0062\times 0.25^2\times 17047\approx 6.7$$

鏡射的 MSE 為 5.62，加上 6.7 約為 12.3，與實測補零的 12.69 相近；差距來自角落像素損失更多權重，以及邊緣附近的亮度不等於整體平均。這是量級估算，不是精確計算。

幾何意義：補零等於假設影像外面還有一圈黑色畫框，視窗滑到邊緣時把黑色也平均進去，邊緣像素被拉暗，與原圖的差異集中在影像外圈。鏡射與複製邊緣都用影像自身的像素補邊，保留了局部亮度，所以 PSNR 幾乎相同（40.63 與 40.65 dB）；補零引入了原本不存在的黑邊，PSNR 因此低約 3.5 dB。

結論：邊界處理會直接影響 PSNR，比較不同濾波器的 PSNR 時必須固定同一種邊界處理。本專案預設使用鏡射，以避免黑邊造成的人為誤差。

3.7.5 客觀評估分析

1. **模糊類濾波器的 PSNR 約 34–37 dB**，屬於差異小、可接受的範圍。相同 3×3 大小下，Gaussian（37.10 dB）高於 Box（34.75 dB），因為高斯核把較大的權重放在中心，對原圖的改變較小；Box 把九個像素等權平均，模糊較重。
2. **核越大、模糊越強、PSNR 越低**：Gaussian 5×5（34.50 dB）低於 Gaussian 3×3（37.10 dB），因為 5×5 平均了更大範圍的像素，細節損失更多。
3. **銳化的 PSNR 較低（28.65 dB）**：銳化放大邊緣的高頻分量，與原圖的差異主要出現在邊緣，MSE 因此比輕度模糊大得多（可對照圖 10 的差異圖）。
4. **Emboss 與邊緣偵測類的 PSNR 很低（約 9–18 dB），但這不代表「品質差」**：這類濾波器的目的是產生浮雕或邊緣圖，輸出本來就不像原圖。以預設的 128 偏移為例，邊緣圖的大部分區域在 128 附近，與原圖亮度分布相差很大，所以 MSE 達數千。對這類濾波器，PSNR 只能說明「與原圖差多少」，不能當成「結果好不好」的指標。
5. **PSNR 的限制**：它逐像素比較數值差，不考慮人眼感知與結構（例如整張圖平移一格，視覺幾乎相同，PSNR 卻可能很低）。因此本報告同時提供並排對照與差異圖，作為量化數字之外的視覺佐證。
6. **證據範圍**：以上數值來自單一張測試影像；不同內容（平滑區域多寡、紋理多寡）會得到不同的 PSNR，絕對數值不宜直接類推到其他影像，但各濾波器之間的**相對大小關係**符合理論預期。

4　AI 協作與完整 Prompt 歷程（AI Audit Trail）

4.1　使用的模型

項目	內容
模型	Claude Sonnet 5.5（Anthropic）
介面	claude.ai，啟用檔案與 Shell 工具，並透過桌面應用程式存取專案資料夾
期間	2026-11-08 02:45 – 2026-11-09（Asia/Taipei）
其他 AI 工具	無（全程只使用 Claude Sonnet 5.5）
完整對話（公開連結）	https://claude.ai/share/c7f80221-4bbc-4509-ade9-f5ee6943d3d8（快照，不含附加截圖）
完整對話（離線備份）	`prompt_log/raw_exports/claude_conversation_2026-11-08_to_11-09.pdf`

完整歷程見儲存庫 `prompt_log/`：每個階段依「原始 Prompt（逐字）→ AI 產出 → 問題 → Refinement → 結果 → 提示工程觀察」記錄，完整對話原文放在 `prompt_log/raw_exports/`。

4.2　協作流程

階段	使用者 Prompt 的策略	AI 的產出
規劃（P1–P3）	設定角色、要求「先不做任何動作」；貼上評分表與繳交規範；逐題回答確認問題	目錄結構、評分對應表、防呆清單；僅規劃、未建立檔案
環境檢查（P4–P5）	授權 AI 直接檢視專案資料夾	發現 OpenCV 5.x 沒有 Haar，在寫程式前先修正
實作（P5–P8）	先短指令（「請遵守規則，並且繼續實作」），再用截圖回報；使用條件式指令（「有問題就修正，無問題就繼續」）	io／color／直方圖、卷積／PSNR／防呆、人臉偵測與測試
部署（P9–P14）	直接貼上終端機輸出	README、Git 與 Streamlit Cloud 疑難排解

4.3　AI 的失誤與修正

#	問題	發現方式	修正
1	AI 假設使用者會裝到 OpenCV 4.x，建議只做 Haar；實際裝到 5.0.0.93，	AI 檢視專案資料夾	降版為 `opencv-python-headless<5`

	沒有 Haar		
2	規劃的 src/ 版面在實作時改為根目錄 imagelab/	AI 自行判斷（部署與 pytest 較單純）	回覆中說明差異與理由
3	直方圖中文標題顯示為方框	使用者截圖	自動偵測中文字型、無字型時退回英文、加入 packages.txt
4	AI 的執行環境無法執行 Windows 虛擬環境、Streamlit 與 Haar	驗證時發現	以替身測試執行器驗證演算法，並明確告知未驗證的部分，由使用者本機實跑
5	指令中的 <你的帳號> 占位符被原樣貼入 git remote	使用者貼上錯誤輸出	git remote set-url 修正
6	AI 無法連線 GitHub、網頁查詢受限	診斷 Repository not found 時	提供讓使用者自行判斷的對照表，不假裝已驗證

這些失誤的共同點是「**預設值未經確認就被使用**」，修正方式都是先檢查實際狀態再下指令。

4.4　完整 Prompt 歷程（依時間序）

以下依時間序記錄開發過程中的每一則 Prompt。每則皆包含：**原始 Prompt（逐字）→ AI 產出摘要 → 問題與修正（Refinement）→ 結果**。使用者在對話中貼上的截圖為附件，此處以文字說明其內容，截圖原檔見儲存庫 docs/images/，並已用於本報告第 3 章。

完整對話原文：公開分享連結（為快照，涵蓋至 P24b；之後的紀錄見下文）；離線備份 PDF 位於儲存庫 prompt_log/raw_exports/。

階段 1　需求分析、規劃與環境建置（P1–P4）

P1　2026-11-08 02:45　需求與目錄結構

原始 Prompt（逐字）

```
請先遵守以下的規則


1. 你是一位資深電腦視覺工程師兼軟體架構師，我需要你完成以下的要求，先不做任何動作，請先了解我們現在該怎麼做，並且要做什麼。
2. 環境：Python 3.12、Streamlit，開發一個互動式影像處理與人臉分析工具開發。
3. 影像載入與基礎解析：支援常見點陣圖格式（至少包含 PNG、JPG/JPEG、BMP）之檔案讀入與即時顯示，正確顯示影像基本中繼資訊（影像解析度、色彩維度、檔案大小等）。
4. 多色彩空間轉換與同步直方圖分析：提供介面切換檢視：原始影像（Original）、RGB 各分量獨立檢視（R/G/B 分
```

離頻道通道圖）、YUV 色彩空間轉換（Y/U/V 亮度與色度分量圖）。

* 同步直方圖繪製：在切換各顯示模式時，需同步動態渲染對應通道的灰階/強度分布直方圖（Histogram），直方圖橫軸需為像素強度（0–255），縱軸為像素數量。

5. 自訂卷積核（Kernel Matrix）空間濾波與量化評估：

* 自訂濾波矩陣：提供使用者動態輸入一組任意數值之二維陣列（例如 3×33\times33×3 或 5×55\times55×5 卷積核，支援邊緣銳化、高斯平滑、浮雕等權重自訂），並可選擇正規化（Normalization）參數。
* 濾波計算與顯示：對輸入影像進行卷積濾波運算，並排或疊加比對濾波前後之視覺成果。
* 量化品質指標（PSNR）計算：以原圖與濾波後影像為基準，即時計算並數值化顯示峰值信噪比（Peak Signal-to-Noise Ratio, PSNR）（以 dB 為單位），並於說明文件中推導/註明計算公式。

6. 人臉偵測（Face Detection）模組：

* 整合經典特徵辨識演算法（如 Haar-Cascade、HOG）或輕量級深層學習檢測器（如 MediaPipe Face Detection、MTCNN、YOLO-Face）。
* 能在影像中標記出所有偵測到的人臉邊界框（Bounding Box），並回傳/標註偵測到的總人臉數量與推論信心度（Confidence Score）。

7. 並且本次作業需要附上版本控制儲存庫，現在請先提供專案目錄的結構就好。

AI 產出摘要

- 依指示**不寫程式、不建立檔案**，只提供目錄結構。
- 設計原則：UI 層（Streamlit）與演算法層（NumPy/OpenCV）分離，演算法模組不 import streamlit，方便單元測試。
- 結構涵蓋 io/、color/、analysis/、filtering/、detection/、ui/、tests/、docs/。
- 提出 5 個待確認問題：人臉偵測器（MediaPipe 或 Haar）、卷積是否手刻、PSNR 通道處理方式、文件格式、Git 託管平台。

提示工程觀察

- 使用者先設定**角色**（資深電腦視覺工程師兼軟體架構師），並用「先不做任何動作」限制輸出範圍，避免一次產出大量未確認的程式。
- 需求以編號列出並附驗收條件（例如直方圖橫軸 0–255），讓後續測試有明確依據。

P2　2026-11-08 03:43　評分表與繳交規範

原始 Prompt（逐字）

附圖：評分標準表（功能完整性與演算法正確性 35%、可執行成果與部署品質 20%、AI 協作歷程 Prompt Log 20%、技術報告說明書與排版 PDF 15%、介面設計與使用者體驗 UX 10%）。文字部分如下：

以下是必須遵守的繳交規範

1. 專案概述與環境建置
2. 可執行連結（Live Demo / Source Code）

* 提供線上即時執行展示連結（如 Hugging Face Spaces、Streamlit Cloud、Google Colab 等，需確認檢閱權限為 Public 開放）。
* 附上版本控制儲存庫（GitHub / GitLab）完整原始碼連結。

註：我應該會另外用 colab 來展示執行成果，這個部份我們可以最後做完再處理， 然後會將我們程式原始碼都推到 github 上面。

3. 程式功能與操作說明書

* 各模組架構圖與資料流說明。
* 功能 1 至 4 之介面操作截圖、功能實測說明與測試案例分析（含邊界條件測試，如極端矩陣輸入、多人臉場景等）。
* PSNR 公式定義與濾波效果的客觀評估分析。

4. AI 協作與完整 Prompt 歷程（AI Audit Trail）

* 完整記錄開發過程中所使用的 AI 模型版本（如 GPT-4o、Claude 3.5 Sonnet 等）。
* 依時間序或模組呈現「原始 Prompt 提問 →\rightarrow→ AI 產出結果 →\rightarrow→ 除錯/調整/重構指令（Refinement）」，展現提示工程（Prompt Engineering）之邏輯推演與除錯過程。

5. 附圖為本次實作的評分表。

AI 產出摘要

- 指出評分表中 Prompt Log（20%）、報告 PDF（15%）、部署（20%）合計 55%，專案不能只有程式碼，因此把 prompt_log/、docs/report/、validation/、README、部署檔案加入目錄結構。
- 製作「評分項目 ↔ 對應交付物 ↔ 關鍵風險」對照表。
- 列出 UX 防呆案例清單：非方陣、偶數尺寸、非數值、正規化除以零、PSNR 的 MSE = 0、影像過大與格式不支援、無人臉。
- 提醒 Prompt Log 要「從現在開始」記錄，並說明格式。

- 建議進行順序，並再問 5 個確認問題。

提示工程觀察

- 使用者把**評分標準直接貼給模型**，讓規劃從「做出功能」轉成「對應配分的交付物」，這是這輪最大的方向修正。
- 使用者同時說明「Colab 最後再做」，用明確的優先順序控制範圍。

P3　2026-11-08 03:55　回答確認問題與環境第一步

原始 Prompt（逐字）

```
1. 先做一種就好，此專案要求即可。
2. 做你覺得最好的那個方法即可。
3. 兩者都顯示。
4. 報告請最後做完後再總整理並且實作。
5. 今天 deadline XD。

請問我第一步是要 uv python install 嗎？
```

AI 產出摘要

- 回答：不一定需要 `uv python install`；真正第一步是安裝 uv 並 `uv init --python 3.12`，缺版本時 uv 會自動下載。給出完整指令（建立專案、`uv add` 依賴、匯出 `requirements.txt`）。
- 依使用者回答做決定：
  - 人臉偵測只做一種 → **改為 OpenCV Haar-Cascade**（先前建議 MediaPipe，理由是 MediaPipe 在 Python 3.12 與雲端環境的相容性風險較高，且今天是截止日，優先低風險）。
  - 卷積 → 純 NumPy 向量化手刻，並用 OpenCV 對照驗證。
  - PSNR → 逐通道平均與亮度 Y 兩者都顯示。
- 因為今天截止，給出時間分配建議，並提醒先保存 Prompt Log。

問題

- 這裡 AI 做了一個**沒有驗證的假設**：認為 OpenCV 內建 Haar 模型檔，所以「不需額外套件」。此假設以 OpenCV 4.x 為前提，沒有先確認使用者會安裝到哪個版本。這在 P4 被證實是錯的（見下）。

P4　2026-11-08 04:01　授權檢視專案資料夾

原始 Prompt（逐字）

好的
C:\Users\eason\Desktop\DIP-mid 這邊的目錄，是讓你確保我有完成或者任何有錯誤的地方，剛剛我已經完成了以上的要求，請跟我說下一步。

AI 產出摘要

- 先請求資料夾存取權限，再讀取 pyproject.toml、.python-version、requirements.txt、Git 狀態與已安裝的 cv2 套件內容。
- 確認 Python 3.12.13、依賴都已安裝、Git 已初始化但無 commit、缺少 .gitignore。
- **發現問題**：使用者安裝到的是 opencv-python-headless 5.0.0.93。檢查型別檔與資料夾後發現：這個版本沒有 CascadeClassifier、沒有 HOGDescriptor，cv2/data/ 內也沒有任何 haarcascade_*.xml。AI 在回覆中承認是自己先前的疏漏。

Refinement（修正指令）

AI 提供的修正（分支改名與第一個 commit 於 P4–P5 之間完成；OpenCV 降版於 P5–P6 之間完成）：

```
uv add "opencv-python-headless<5"
uv run python -c "import cv2; ... Haar OK ..."
uv export --no-dev --no-hashes -o requirements.txt
git branch -M main
```

並提供 .gitignore 內容與第一個 commit 的指令。

最終結果

- P5 檢查時：分支已是 main、已有第一個 commit，但 **OpenCV 仍是 5.0.0.93、.gitignore 不存在**（降版尚未完成）。AI 先自行補上 .gitignore，並繼續實作不依賴 Haar 的模組，再次提醒降版。
- P6 檢查時確認：OpenCV 已是 **4.14.0.94**，cv2/data/ 有完整的 Haar XML，pyproject.toml 已寫入 opencv-python-headless<5，requirements.txt 已重新匯出。

提示工程觀察

- 使用者**讓 AI 直接檢查專案資料夾**，而不是自己描述狀況，這讓 AI 能在寫任何程式前先發現環境錯誤，避免做出無法執行的人臉偵測模組。
- 這是整個專案中最重要的一次「先驗證環境，再寫程式」。

階段 2　影像載入與色彩空間（P5–P6）

P5　2026-11-08 04:04　檢查並繼續

原始 Prompt（逐字）

```
OK，請檢查，並繼續下一步
```

AI 產出摘要

- 再次檢查資料夾：確認 OpenCV 降版與 commit 已完成，之後才開始寫程式。
- 查閱已安裝的 Streamlit 原始碼，確認 `st.image` 的寬度參數寫法（`width="stretch"`），避免使用已棄用的 `use_container_width`。
- 建立並實作：
  - `imagelab/io/loader.py`：依檔案**內容**判斷格式（非副檔名）、套用 EXIF 方向、RGBA 合成白底、超大圖與損毀檔的友善錯誤。
  - `imagelab/io/metadata.py`：解析度、通道數、檔案大小等。
  - `imagelab/color/channels.py`、`yuv.py`：RGB 分離與 BT.601 全範圍 YUV 轉換（公式寫在檔案註解）。
  - `imagelab/analysis/histogram.py`：橫軸 0–255、縱軸像素數。
  - `imagelab/ui/`：側邊欄、影像資訊分頁、色彩與直方圖分頁（影像與直方圖由同一次 render 的同一份資料算出，保證同步）。
  - `tests/`：載入、色彩、直方圖共 19 個測試，含空檔案、損毀檔、RGBA、超大圖、uint8 檢查等邊界條件。

問題與限制

- **版面偏離**：先前規劃用 `src/imagelab/`，實作時改成根目錄 `imagelab/`，AI 在回覆中說明理由（雲端部署與 pytest 不需額外設定匯入路徑）。
- **驗證限制**：AI 的 Linux 環境無法執行使用者的 Windows `.venv`，也沒有網路可安裝 pytest。AI 改寫一個「最小 pytest 替身執行器」直接呼叫測試函式，19 個全過；並**明確告知這不等於真正的 pytest**、Streamlit 介面也沒有實跑，要求使用者在本機驗證。

最終結果

使用者在本機執行後提供截圖（P6），確認影像資訊、Original、RGB、YUV 三種模式與直方圖都正常顯示。

P6　2026-11-08 04:19　中文字體與進度評估

原始 Prompt（逐字）

附圖：4 張介面截圖（影像資訊、Original 模式、RGB 分量、YUV 分量）。文字如下：

> 記得處理中文字體的顯示問題，其他請評估當前狀態並且給出下一步執行的方向。

問題（由截圖發現）

直方圖標題的中文字顯示成方框（如「□□□□」）。原因：matplotlib 預設字型 DejaVu Sans 沒有中文字。

AI 產出摘要（Refinement）

- 新增啟動時**自動偵測系統中文字型**（涵蓋 Windows 微軟正黑體、macOS、Linux Noto CJK 等），設為優先字型。
- 偵測不到時，圖表標題**自動退回英文**（`title_en`），並在畫面顯示提示，不會再出現方框。
- 新增 `packages.txt`（內容 `fonts-noto-cjk`），讓雲端部署也能安裝中文字型。
- 設定 `axes.unicode_minus = False`，避免負號變方框。
- 新增 3 個字型退回測試（共 22 個測試通過，替身執行器）。
- 同時檢查專案狀態：確認 OpenCV 4.14.0.94 降版完成、Haar XML 已就位。
- 提出下一步（卷積濾波）的設計提案：核輸入與範本、正規化、邊界處理、防呆、PSNR 設計。

最終結果

使用者在 P8 附上的截圖顯示直方圖標題「Y 直方圖」「U 直方圖」「V 直方圖」已正常顯示中文。

提示工程觀察

- 使用者用**截圖**回報視覺問題，比文字描述更精確，AI 能直接定位到「中文標題變方框」。
- 使用者把兩件事合在一個 Prompt：「修 bug」與「評估狀態並給出方向」，讓 AI 在修完問題後主動回報進度，而不是等待下一個指令。

階段 3　卷積濾波與 PSNR（P6–P8）

P6（設計提案）　2026-11-08 04:19

P6 的原始 Prompt 見 02_io_and_color.md。AI 在修完字型、評估進度之後，提出本階段的設計，供使用者確認：

項目	提案
核輸入	文字框輸入，一列一行，數字以空白或逗號分隔；9 個預設範本（Identity、Box、高斯 3×3 與 5×5、銳化、浮雕、Sobel X/Y、Laplacian）
正規化	不處理／除以總和；總和為 0 時略過並提示
卷積演算法	純 NumPy 向量化；邊界可選補零、鏡射、複製邊緣；測試與 `cv2.filter2D` 對照（真正的卷積要先翻轉核）
防呆	非方陣、偶數、空白、非數字、NaN/無限大，指出哪一列哪一格；核過大、全零核
顯示	濾波前後並排，另附差異圖
PSNR	公式 $10 \cdot \log10(255^2 \,/\, \mathrm{MSE})$；顯示逐通道與亮度 Y；兩圖完全相同時顯示 ∞

P7　2026-11-08 04:33　實作

原始 Prompt（逐字）

好，那麼請遵守規則，並且繼續實作。

AI 產出摘要

新增與修改的檔案：

檔案	內容
`imagelab/filtering/convolution.py`	向量化卷積：核旋轉 180° 後以 k^2 次整張影像運算累加，不逐像素迴圈
`imagelab/filtering/kernels.py`	預設範本與正規化（總和為 0 時回傳「略過」訊息）
`imagelab/filtering/metrics.py`	MSE、PSNR、逐通道與 Y 的報告
`imagelab/validation/`	矩陣解析（支援全形逗號、Unicode 負號、；分列）、尺寸與數值檢查、警告
`imagelab/ui/tab_filter.py`	第三分頁
`docs/psnr.md`	PSNR 公式的代數與幾何推導、數值驗算、判讀表
`docs/convolution.md`	為什麼實作時要翻轉核的推導、邊界處理、正規化
`tests/`	卷積（對照教科書定義與 OpenCV）、輸入驗證、PSNR 的

	測試

關鍵設計決定（AI 在回覆中標明，供寫報告時說明）：

1. 「R/G/B 通道平均」定義為**三個通道 PSNR（dB）的算術平均**，並非先平均 MSE 再換算。
2. Sobel、Laplacian 範本預設勾選「加 128 偏移」，否則負值被截成 0，畫面幾乎全黑。
3. 先轉 `float64` 再相減，避免 `uint8` 溢位（`0 − 255` 變成 `1`）。

驗證結果與限制

- 替身執行器：**62 個通過、0 失敗、0 略過**，其中包含與 OpenCV `filter2D` 的對照（三種邊界、3×3 與 5×5 的非對稱核，皆一致）。
- 效能：800×533 影像濾波約 20 ms；約 2000 萬像素、5×5 核約 3 秒。
- 限制：以上為替身執行器的結果，**不是真正的 pytest**；Streamlit 介面 AI 沒有實跑。

提示工程觀察

- 這次使用者只給一句話：「請遵守規則，並且繼續實作」。能這麼簡短，是因為前面已經把**需求、評分標準、設計提案**都確認過，上下文足夠，AI 不需要再問問題。
- 「遵守規則」指回 P1 的限制與 P2 的評分標準（防呆、公式註明、邊界條件測試）。

P8（濾波分頁驗收）　2026-11-08 04:48

P8 的原始 Prompt 與其他部分見 04_face_detection.md。

使用者附上的第三張截圖顯示：預設範本「Gaussian 高斯平滑 3×3」、勾選正規化、鏡射邊界時，**PSNR（亮度 Y）= 40.63 dB、R/G/B 通道平均 = 40.24 dB、MSE（亮度 Y）= 5.62**；左側「實際使用的核」顯示 0.0625、0.125、0.25，總和為 1，與正規化預期一致。AI 據此判定濾波分頁正常，進入下一步。

階段 4　人臉偵測（P8）

P8　2026-11-08 04:48　進度確認並繼續

原始 Prompt（逐字）

附圖：3 張介面截圖（影像資訊、YUV 分量、卷積濾波與 PSNR 分頁）。文字如下：

```
目前進度，請一樣看一下目前進度，如有問題就修正，無問題就繼續
```

AI 產出摘要

1. 先檢查資料夾：Git 有 2 個 commit，濾波與驗證相關檔案尚未 commit；OpenCV 4.14.0.94 與 Haar XML 都在；`assets/samples/` 是空的。
2. 判斷截圖無問題（中文字型已修好、PSNR 數值合理），進入人臉偵測。
3. **查證 API**：AI 自己的環境是 OpenCV 5.0，沒有 `CascadeClassifier`，因此改讀使用者 venv 內 OpenCV 4.14 的型別檔，確認 `detectMultiScale3(..., outputRejectLevels=True)` 的簽名與回傳值。
4. 實作：
   - `imagelab/detection/haar_detector.py`：OpenCV 內建 `haarcascade_frontalface_default.xml`；超過 1280 像素的圖先縮小再偵測，框換算回原圖座標；影像比偵測視窗小時直接回傳空清單；對分類器加上鎖（多使用者連線時不是執行緒安全）。
   - `imagelab/detection/draw.py`：邊界框、「#編號 信心度」標籤，線寬與字體隨影像大小縮放，框貼近邊緣時標籤不會跑出畫面。
   - `imagelab/ui/tab_face.py`：可調 scaleFactor、minNeighbors、最小人臉尺寸；顯示總人數、最高信心度、每張臉的位置與信心度表格、人臉裁切圖；無人臉時給出調整建議。
   - `tests/test_detection.py`：空白圖、極小圖、最小尺寸大於影像、參數錯誤、畫框不改動輸入、信心度單調等邊界條件。

信心度的設計與限制

Haar 回傳的是最後一層分類器的**層級權重**，沒有上限，也不是機率。AI 用 `confidence = 1 − e^(−w/2)` 壓到 0–1，並同時顯示原始權重；回覆與介面都註明這是**啟發式正規化**，只適合比較同一張影像中各框的相對可信度。

問題與限制

- **無法實跑**：AI 的環境沒有 Haar，所以偵測效果**沒有實際驗證**。AI 明確告知「最可能出問題的地方就在這裡」，並要求使用者在本機跑 pytest 與介面。
- 替身執行器的結果：**68 個通過、4 個略過**（略過的是需要 Haar 的測試）。
- 過程中 AI 的暫存環境重置，替身執行器檔案消失，AI 重新建立後再跑一次。

使用者驗證

使用者實測（截圖存於 `docs/images/face_*.png`，報告第 4.4.2 節）：

情境	結果
預設參數（1.10、5、30）	偵測到 3 張臉（含兩張失焦），信心度 0.993、0.966、0.624
minNeighbors = 1	仍為 3 張臉，位置與信心度和預設相同，沒有多出誤判框

minNeighbors = 12	只剩 2 張，中間較大的清楚人臉不再被保留（依表格列數與綠框數判讀）
最小人臉尺寸調大（約 300）	偵測 0 張
沒有人臉的影像	偵測 0 張，不當機

- 使用者本機 `uv run pytest -q`：**71 passed、1 skipped**（P16），其中含需要 Haar 模型的偵測測試；略過的是選用的真實影像測試。

提示工程觀察

- 「如有問題就修正，無問題就繼續」這種**條件式指令**，讓 AI 自己先審查再決定下一步，而不是機械式地往下做。
- 使用者用的測試圖剛好有一張清晰與兩張失焦的臉，AI 把它列為「多人臉與模糊場景」的邊界測試案例。

階段 5　README、GitHub 與部署（P9–P14）

P9　2026-11-08 21:46　README 與部署

原始 Prompt（逐字）

```
好 來吧
```

AI 產出摘要

- 檢查資料夾：三個 commit 皆已完成、工作區乾淨、尚未設定遠端，`assets/samples/` 與 `docs/images/` 仍無內容。
- 撰寫 `README.md`：連結表、功能表、uv 與 pip 兩種安裝方式、測試、專案結構、公式、輸入防呆表、Streamlit Cloud 部署步驟（含 Python 3.12 設定）、已知限制、AI 協作聲明。
- 修改 `pyproject.toml` 的預設描述；提醒 `git rm main.py`。
- 提供推上 GitHub 與部署的步驟。

P10　2026-11-09 15:07　`git remote add` 失敗

原始 Prompt（逐字）

```
(dip-mid) PS C:\Users\eason\Desktop\DIP-mid> git remote add origin https://github.com/koraqq/DIP-mid.git
error: remote origin already exists.
```

AI 產出摘要與問題

- AI 檢查專案的 `.git` 設定，發現 `origin` 指向 `https://github.com/<你的帳號>/DIP-mid.git`。

- **原因（AI 自己的疏失）**：先前 AI 在回覆中的指令使用了 <你的帳號> 占位符，使用者原樣貼上執行，導致 origin 指向一個無效網址。
- Refinement：改用 `git remote set-url origin ...` 修改既有 origin，而不是再新增。

P11　2026-11-09 15:08　Repository not found

原始 Prompt（逐字）

```
解決以下的問題


(dip-mid) PS C:\Users\eason\Desktop\DIP-mid> git remote set-url origin https://github.com/koraqq/DIP-
mid.git
(dip-mid) PS C:\Users\eason\Desktop\DIP-mid> git remote -v
origin  https://github.com/koraqq/DIP-mid.git (fetch)
origin  https://github.com/koraqq/DIP-mid.git (push)
(dip-mid) PS C:\Users\eason\Desktop\DIP-mid> git push -u origin main
remote: Repository not found.
fatal: repository 'https://github.com/koraqq/DIP-mid.git/' not found
```

AI 產出摘要與限制

- AI 想從外部確認儲存庫是否存在，但網頁查詢被限流，從 AI 的環境用 git 與 GitHub API 也無法連線（網路被擋）。**AI 沒有假裝已驗證**，如實告知。
- 改提供判斷表：請使用者用瀏覽器開 `https://github.com/koraqq/DIP-mid`，依看到的結果（404、私有、空儲存庫）分辨三種原因（儲存庫未建立或帳號名稱錯、私有且帳號驗證錯誤、登入問題），並附上建立儲存庫的步驟（Public、不勾選 README/.gitignore/license）與清除 Windows 憑證的指令。

P12　2026-11-09 15:11　清除憑證後推送

原始 Prompt（逐字）

```
(dip-mid) PS C:\Users\eason\Desktop\DIP-mid> git remote set-url origin https://github.com/koraqq/DIP-
mid.git
(dip-mid) PS C:\Users\eason\Desktop\DIP-mid> cmdkey /delete:git:https://github.com

CMDKEY: Credential deleted successfully.
(dip-mid) PS C:\Users\eason\Desktop\DIP-mid> git push -u origin main
branch 'main' set up to track 'origin/main'.
Everything up-to-date

這邊應該好了
```

AI 產出摘要

- 輸出 `Everything up-to-date` 不是第一次推送到全新儲存庫的典型結果，AI 先檢查本地記錄：`HEAD` 與 `origin/main` 都是 `92271a8`，沒有未推送的 commit。
- AI **如實說明不確定**遠端內容是這次推上去的，還是更早就在那裡，並要求使用者用瀏覽器確認：儲存庫為 **Public**、檔案完整、最新 commit 正確。
- 提供 Streamlit Cloud 的部署步驟，強調 **Advanced settings 要選 Python 3.12**。

P13　2026-11-09 15:58　部署完成

原始 Prompt（逐字）

```
https://dip-mid-tb8vrujcpzjqkwkc5onscz.streamlit.app/
好了
```

AI 產出摘要

- 將部署網址與 GitHub 連結填入 `README.md`（Colab 連結保留占位符）；順手修掉 GitHub 連結的反引號，使其可點擊。
- 嘗試開啟部署網址被網站的 robots 規則拒絕，AI 如實說明**無法自行驗收線上版**，提供驗收清單請使用者用無痕視窗檢查：是否不登入就能開、中文直方圖標題是否正常（雲端是否裝到 `fonts-noto-cjk`）、PSNR 是否與本機一致、第四分頁是否出錯。

P14　2026-11-09 16:00　開始整理 Prompt Log

原始 Prompt（逐字）

附圖：線上展示畫面截圖（標題「Image Lab：影像處理與人臉分析」、側邊欄上傳區、「請從左側上傳一張 PNG / JPG / BMP 影像開始。」提示，頁面右上角有 Fork 與 GitHub 圖示）。文字如下：

```
OK了 那就開始 Prompt Log
```

結果

截圖顯示線上版已能正常載入首頁。

使用者驗證

線上版四個分頁可正常開啟與操作。

提示工程觀察

- 這一段的 Prompt 多為**終端機輸出直接貼上**，AI 能從錯誤訊息直接定位原因（占位符、憑證、可見性），比文字描述更有效。
- 兩次出現的 AI 疏失（OpenCV 版本假設、占位符被當成真實網址）有共同點：**提供了未經確認就被直接使用的預設值**，修正方式都是「先檢查實際狀態再下指令」。

階段 6　報告、補件與繳交前檢查（P15–P28）

P15　2026-11-09 17:19　檢查補件並開始報告

原始 Prompt（逐字）

```
我將一些結果都放進去了 如有缺失請跟我說還需要什麼，並且來開始報告
```

AI 產出摘要

1. **檢查補件**：發現 `docs/images/` 有 13 張截圖，但 `prompt_log` 內的三個待補處與 `raw_exports/` 仍為空；`assets/samples/` 仍沒有範例影像；尚無防呆畫面與人臉偵測調參的截圖。
2. **判讀截圖**（AI 無法直接檢視使用者資料夾內的圖片，改用間接方式）：
   - 色彩分頁：用像素顏色辨識模式（RGB 模式有大量紅／綠／藍直方圖色；YUV 模式有 Y／U／V 直方圖色且影像為灰階）。
   - 濾波分頁：用 OCR（僅英文）讀取預設範本名稱與核矩陣；並用「MSE 與 PSNR 必須互相吻合」交叉驗證讀到的數字（`PSNR = 10 · log10(65025 / MSE)`），8 張都通過。截圖對應預設範本的順序（Box、Gaussian 3×3、Gaussian 5×5、Sharpen、Emboss、Sobel X、Sobel Y、Laplacian）。
3. **產出**：`docs/report/` 下的 `report.md`（報告原稿）、`make_diagrams.py`（架構圖與資料流圖）、`build_report.py`（pandoc → docx → LibreOffice → PDF 的建置腳本）、`figures/`（以有意義檔名複製的截圖與兩張圖）、`report.pdf`（24 頁，A4）。

問題與修正（Refinement）

問題	發現方式	修正
6 個表格寬度為 0，LibreOffice 依內容撐開，導致 9 頁內容超出右邊界	把每頁轉成圖，計算內容的左右邊界	後處理時將所有表格固定為文字區寬度，依內容長度分配欄寬，長測試名稱與網址可在 `_` / `.` 處換行
LibreOffice 設定檔寫入專案資料夾失敗	轉檔時出現 `DeploymentException`	改用系統暫存目錄
封面標題最後一個字「具」單獨換	比對每頁的文字量與內容範圍	標題字級 24 → 20 pt，內文行距

行；第 21 頁只剩一個孤立項目		1.35 → 1.25（24 頁、無孤立頁）
報告中一句「差異圖也顯示變化集中在邊緣」沒有依據（AI 沒看過該圖）	寫完自行審查	改為「主要出現在邊緣，可對照圖 10 的差異圖」

限制（如實記錄）

- AI **無法看到 PDF 的實際畫面**，版面僅以量化方式檢查（頁數、邊界、空白、字型嵌入、圖片數量）；請使用者逐頁目視確認。
- 截圖的對應關係與 PSNR 數值來自 OCR 與順序推論，Y 的 PSNR 與 MSE 已互相驗證，但 R/G/B 平均值與「邊界處理、正規化、偏移」等設定無法從截圖確認，需使用者核對。
- 報告有 9 處以黃底標示的「待補」標記，需使用者提供資料後重新建置。

提示工程觀察

- 使用者把「檢查缺什麼」與「開始報告」合成一個 Prompt，讓 AI 先盤點證據再動工，而不是先寫出一份缺料的報告。
- 當 AI 無法直接讀取圖片時，改用「可驗證的間接證據」（顏色統計、OCR、MSE 與 PSNR 的一致性）取代猜測，並在報告中保留需使用者確認的標記。

P16　2026-11-09 20:47　補入資料、取消 Colab、詢問防呆畫面做法

原始 Prompt（逐字）

附圖：報告 PDF 待補清單中「防呆畫面」那一列的截圖（內容為「截圖各一張：非方陣、偶數尺寸、含非數字、Sobel 勾選正規化的「已略過」提示、Identity 的 PSNR 顯示 ∞」）。文字如下：

```
streamlit 已經做完了 那就不用 colab 了 因為只是要看執行的成果
作者：林奕丞，C113151143
這個部分是作業，不會拿去做任何營利，之後確保繳交作業後就會改回私人


(dip-mid) PS C:\Users\eason\Desktop\DIP-mid> uv run pytest -q
.............................s.....................................
[100%]
71 passed, 1 skipped in 1.69s


然後防呆畫面可以詳細解釋一下嗎？
我不確定怎麼做。
```

AI 產出摘要

- 將作者、pytest 結果（71 passed、1 skipped）、測試影像用途聲明填入報告；移除 Colab（作業只要求「可執行的線上展示」，Streamlit Cloud 已滿足），並同步修改 README。
- **pytest 結果的意義**：先前 AI 的環境無法執行 Haar，人臉偵測測試只能略過；使用者本機 71 個通過、只略過 1 個選用測試，代表含 Haar 的偵測測試（空白圖、極小圖、最小尺寸大於影像）在真實環境通過，補上先前「未驗證」的缺口。
- 詳細說明 5 張防呆畫面的操作步驟、要輸入的內容、預期出現的訊息與截圖命名（見對話回覆）。
- 提醒：改成私人儲存庫後，Streamlit 公開展示與老師檢閱可能失效，應在評分完成後再改。

提示工程觀察

- 使用者在不確定如何操作時，直接把「待補清單」的那一列截圖貼回來並說「我不確定怎麼做」，讓 AI 能針對那一項給出具體步驟，而不是泛泛而談。

P17　2026-11-09 21:59　防呆截圖完成與檔名整理

原始 Prompt（逐字）

```
我現在已經完成了那五張照片的防呆畫面 以及命名了 可以的話 其他照片也幫我修正一下名稱好了
```

AI 產出摘要

- 將 `docs/images/` 中 13 張 `Screenshot ...` 依內容改名為 `ui_info`、`ui_color_*`、`ui_filter_*`、`ui_face_default`（與報告用複本同名，已逐檔比對內容一致）；只改名、不改內容、不刪除。
- 用 OCR（僅英文）核對 5 張防呆截圖：矩陣內容（2×3、2×2、abc）與預設範本名稱都與預期相符；中文錯誤訊息 OCR 無法辨識，依程式碼中的訊息撰寫圖說。
- 將 5 張防呆截圖放進報告 4.3.4 節，圖號順延（人臉偵測圖由圖 15 變為圖 20），並在邊界條件表中補上對應圖號與 Identity 一列。

觀察與限制

- 防呆截圖中的邊界處理顯示為「補零（zero）」，而程式預設為「鏡射（reflect）」，推測使用者在截圖時改過設定；這也為先前「Gaussian 3×3 為何是 37.10 dB 而非 40.63 dB」提供佐證，但仍待使用者確認。
- 第 4 張（Sobel X）的 MSE 約為 17047，高於先前 Sobel X 的 8058，推測此張未啟用「加 128 偏移」；因此圖說不描述偏移設定。

P18　2026-11-09 22:13　回答補件問題

原始 Prompt（逐字）

1. 應該是按下去後就拍了 沒有做任何其他特別的調整，如果還需要截圖什麼再跟我說要截圖什麼，要怎麼做。
2. 第二個問題也是一樣，我記得每個截圖都會有顯示有沒有補 0 等等的問題
3. 沒有其他 AI 工具了
4. "C:\Users\eason\Desktop\DIP-mid\docs\images\ui_face_default.png" 這個預設的臉部辨識部分還要其他的調整跟我說怎麼調整，要截圖什麼。

AI 產出摘要

- 以 OCR 檢查 8 張濾波截圖的邊界處理：5 張可辨識為「補零（zero）」，其餘 3 張 OCR 未讀到，但與其他張為同一連續操作。AI 嘗試用像素分析判斷勾選框狀態，因截圖為淺色主題而失敗，改以「使用者未額外調整，所以設定為各預設範本的預設值」處理，並在報告中如此註明。
- 以 PSNR 數值反推：補零使影像邊緣變暗，額外 MSE 約 = 邊框像素比例 × 損失權重² × E[Y²] ≈ 0.00624 × 0.0625 × 17047 ≈ 6.7，加上鏡射時的 5.62 約為 12.3，與觀察到的 12.69 相符，支持「37.10 dB 與 40.63 dB 的差異來自邊界處理」。此為估算，已請使用者拍攝三種邊界處理的對照畫面以實際驗證。
- 人臉偵測截圖：影像中可見 2 個明顯的綠框，畫面左側的數字讀到「3」、最高信心度 0.99；AI 推測偵測到 3 張臉，但此截圖沒有拍到參數滑桿與表格，請使用者確認並建議重拍。
- 「其他 AI 工具：無」寫入報告與 `00_models_used.md`。

限制

- AI 無法讀取中文，也無法直接檢視圖片，數字與狀態都是間接推斷，所以報告中仍保留需使用者確認的「待補」標記。

P19　2026-11-09 22:31　邊界處理對照與人臉偵測截圖完成

原始 Prompt（逐字）

AB 都完成了

AI 產出摘要

- 以 OCR 讀取 3 張邊界處理對照截圖：鏡射 40.63 dB、補零 37.10 dB、複製邊緣 40.65 dB；PSNR 與 MSE 可互相換算，與先前的代數估算相符（補零額外增加的 MSE 約 6.7，5.62 + 6.7 ≈ 12.3，實測 12.69）。
- 人臉偵測表格以「信心度 = 1 − e^(−w/2)」交叉驗證 OCR 結果：權重 10.022 → 0.993、6.789 → 0.966，與畫面標籤相符；第 3 列權重 1.935 → 0.620（OCR 曾讀成 0.626，以權重換算為準，並請使用者核對）。
- 報告新增 4.4.2 的實測表格與調參截圖（圖 20–24）與第 5.4 節（邊界處理對 PSNR 的影響，圖 25–27）。

問題與限制

- AI 無法可靠讀出調參截圖中的人臉數量與滑桿設定值（OCR 讀不到中文標籤與部分數字），報告中保留「待補」標記，請使用者提供。
- 低 minNeighbors 與高 minNeighbors 兩張截圖，綠框偵測結果與預期（低 → 較多、高 → 較少）不易從截圖間接判斷，因此不下結論。

提示工程觀察

- 這一輪只需要一句話回報，是因為前一輪已明確列出「要拍什麼、檔名、做法」，使用者能直接照做，AI 再依檔名取得資料。

P20–P22　2026-11-09 22:41 – 22:53　補數值與重拍調參截圖

原始 Prompt（逐字）

P20（22:41，附 Gaussian 3×3 預設狀態截圖）：

```
1. 圖 21、圖 22（`minNeighbors` 調低與調高）：我讀不出滑桿的設定值，也看不出偵測到幾張臉。請告訴我：這兩張各自把 `minNeighbors` 設成多少、偵測到幾張臉，以及跟預設的 3 張比有什麼差別。跟我說哪個檔名就好 你說圖 21 22 我不知道是什麼圖


2. 信心度的部分 1 是 0.993 2 是 0.966 3 是 0.624
3. 是鏡射的樣子
```

P21（22:50）：

```
那不然這兩張照片重新截圖一下試試吧
既然有點模糊，你給我所有的數值來截圖 這樣也不怕你不知道了
```

P22（22:53）：

```
已完成
```

AI 產出摘要

- 指出「圖 21、22」對應的檔名，並改成「把設定值寫進指示」的做法：重新整理頁面回到預設值，用滑桿的 Home／End 鍵精準調到 minNeighbors = 1 與 12，其餘參數不動。
- 使用者確認第 3 列信心度為 0.624；AI 先前由權重 1.935 算出 0.620、OCR 曾讀成 0.626，三者皆不準，以使用者讀到的值為準。
- 使用者截圖時沿用了舊檔名（face_2_neighbors_low.png、face_3_neighbors_high.png），AI 以檔案修改時間與解析度變化確認為新截圖。

- OCR 結果：minNeighbors = 1 時為 3 張臉，位置、大小、信心度與預設相同；minNeighbors = 12 時表格只剩 2 列（左側失焦的臉與右側的臉），中間較大的臉未保留。
- 報告 4.4.2 節補上結果與觀察，並更正第 5.3 節「拍攝時未做其他調整」的不準確說法（改為「截圖顯示補零，程式預設為鏡射」）。

限制

- 滑桿設定值與人臉數量無法直接從畫面讀出（OCR 讀不到中文標籤），以表格列數、綠框數量與檔名交叉推斷。權重的小數第 3 位 OCR 不穩定（10.021／10.022、1.956／1.965），所以報告以小數第 2 位呈現。
- 「大臉被高 minNeighbors 濾掉」的原因為推測，已在報告中如此標明。

P23–P24　2026-11-09 23:01 – 23:05　PDF 預覽問題與加入目錄

原始 Prompt（逐字）

P23（23:01，附 GitHub 上 `report.pdf` 預覽失敗的截圖，畫面顯示「Unable to render code block」）：

```
是
GitHub 儲存庫是公開的
點開 pdf 會說程式無法渲染
線上版是能開的
```

P24（23:05，附報告封面截圖）：

```
直接下載是能看的 然後我希望你將章節導覽新增一頁目錄 這樣我們能更直覺的知道。
5.1 跟 5.4 確實都是正常的
```

AI 產出摘要

- **PDF 預覽失敗**：先驗證推上去的版本——與本機檔案位元組完全一致、31 頁、檔案結構無錯誤、文字與圖片皆可讀出、無 Git LFS 影響，判斷為 GitHub 預覽器的問題，而非檔案損壞（推測 Brave Shields 也可能是原因，無法從 AI 端確認）；在 README 加入可直接下載的連結。使用者之後確認「直接下載能看」。
- **目錄**：將封面的「章節導覽」改為獨立的目錄頁（第 2 頁），列出 7 章、17 個小節與參考資料共 27 條，附點線與頁碼。頁碼採兩輪建置：先建置取得各標題實際所在頁，再填入並重建，直到頁碼穩定。

問題與修正（Refinement）

問題	發現方式	修正
「5.5 客觀評估分析」找不到所在頁	建置腳本報錯	該標題緊接編號清單，PDF 文字把標題的編號與文字拆開（中間夾了

		清單的「1.」）；比對改為先找完整標題，找不到再用不含編號的標題文字
目錄頁碼驗證腳本顯示不一致	驗證時	腳本多讀到頁尾的頁碼「2」，前 27 條實際一致，為驗證腳本本身的誤差

限制

- AI 看不到 PDF 的實際畫面，頁碼正確性以「每個標題的文字確實出現在目錄標示的頁面」來交叉驗證。

P24b　2026-11-09 23:10　補完 Prompt Log、詢問分享連結

原始 Prompt（逐字）

```
補一 已經完成
完整對話我是打算直接開一個存取連結，你認為呢？
```

AI 產出摘要

- 確認使用者補上的 `05_git_deployment.md` 已生效；清掉標題殘留的「（待補）」。
- 查閱 Anthropic 說明文件後回覆分享連結的特性：是**建立當下的快照**（之後的訊息不含，需取消後重新分享才會更新）；**附加的檔案不會出現**（本專案貼的截圖看不到）；可隨時關閉；任何有連結者可看。
- 建議不要只放連結：等對話結束再建立、另存 PDF 作離線備份、把連結寫進文件。並說明 AI 無法自行匯出逐字對話（工具輸出已被清除，重建會不完整）。

P25　2026-11-09 23:19　建立分享連結與備份 PDF

原始 Prompt（逐字）

```
https://claude.ai/share/c7f80221-4bbc-4509-ade9-f5ee6943d3d8
以建立連結，也確保沒有登入狀態下能看見了


也成功儲存成 pdf 了
```

AI 產出摘要

- 將連結寫入 `raw_exports/README.md`、`prompt_log/README.md` 與報告 6.1 節。
- AI 無法開啟分享頁（動態載入），因此無法自行驗證連結內容。

- **檢查備份 PDF，發現不完整**：28 頁，開頭是第 10 輪（P10，2026-11-09 15:07 的 `git remote add` 錯誤），沒有 P1–P9（需求、評分表、環境、各模組實作）；頁尾網址為 `claude.ai/chat/...`，是從原本的對話頁列印，而不是分享頁。推測原因是對話頁採延遲載入，列印時只輸出已載入的較新訊息。

Refinement

請使用者開啟分享連結，**先捲到最上方並等待載入完成、再捲到最底部**，確認第一則訊息為 P1 後再列印為 PDF，並檢查頁數與開頭內容。

提示工程觀察

- 「已成功儲存成 PDF」不等於「內容完整」。AI 以檔案內容（關鍵詞出現次數、第一頁開頭與最後一頁結尾）檢查，而不是只確認檔案存在。

P26　2026-11-09 23:26　同一連結與詢問是否完成

原始 Prompt（逐字）

```
https://claude.ai/share/c7f80221-4bbc-4509-ade9-f5ee6943d3d8
新連結


然後目前是已經完成了嗎？
```

AI 產出摘要

- 指出連結與先前相同；檢查使用者重存的 PDF（30 頁）：第一頁仍從 P10 開始，頁尾網址為 `claude.ai/chat/...`，代表仍是從原對話頁列印，P1–P9 未收錄。
- 列出整體完成度表，指出唯一未完成的是備份 PDF，並再次說明正確作法（用無痕視窗開分享頁、先捲到最上方再捲到底、確認頁尾網址為 `claude.ai/share/...`）。

P27　2026-11-09 23:30　準備繳交

原始 Prompt（逐字）

```
好了 如果沒問題我就繳交了喔
```

AI 產出摘要

- 驗證新的備份 PDF：66 頁、頁尾網址為分享頁、第一頁起於 P1、含評分表與繳交規範、止於分享快照建立當時的最後一則回覆（P25 之前）。
- 發現 Prompt Log 漏記 23:10、23:26、23:30 三則 Prompt，補上本段紀錄。

提示工程觀察

- 「存好了」與「內容完整」是兩件事：AI 兩次都以 PDF 內容（第一頁開頭、頁尾網址、關鍵詞）驗證，第一次與第二次發現不完整，第三次才通過；使用者因此在繳交前補齊了證據。

P27b　2026-11-09 23:34　Ctrl+Z 誤復原與最後檢查

原始 Prompt（逐字）

```
可能我剛剛按到 ctrl+z 他就自己跑回下載了
那麼請再檢查一次，確認沒問題我就繳交了
```

AI 產出摘要

- 做了唯讀的完整檢查：備份 PDF 已回到資料夾（66 頁、來自分享頁、起於 P1，文字內容與 Git 中的版本相同，只差列印時間）、必要檔案齊全、無待補標記、沒有追蹤到雜物；發現 README 仍寫報告「31 頁」，改為 32 頁。
- 先前 AI 嘗試以 `git restore` 還原被移走的 PDF 時，動作被系統的安全檢查擋下；AI 沒有改用其他方式繞過，而是說明情況並請使用者自行還原。原因後來證實是使用者的 Ctrl+Z。

P28　2026-11-09 23:40　依繳交規範重整報告

原始 Prompt（逐字）

附圖：課程「繳交規範與驗收項目」（僅接受 PDF；檔名規則「學號_姓名_期中專題.pdf」；報告必備 Chapter 1–4：專案概述與環境建置〔含開發語言版本、`requirements.txt` 條目說明〕、可執行連結、程式功能與操作說明書、AI 協作與完整 Prompt 歷程）。文字如下：

```
欸對因為檔案只要交 pdf 就好 所以可以的話幫我檢查每一頁是否有清晰的圖還有 pdf 還有原始碼的連結等等
就是這樣
```

AI 產出摘要

對照規範檢查現有報告，發現 5 處不符並修正：

#	不符之處	修正
1	檔名未依「學號_姓名_期中專題.pdf」	建置時另存 C113151143_林奕丞_期中專題.pdf
2	章節為 7 章，與規定的 4 章不同	重整為 Chapter 1–4：原「系統架構」「功能說明」「PSNR 評估」併入 Chapter 3，AI 協作成為

		Chapter 4，結論與參考資料列於其後
3	Chapter 1 缺少 `requirements.txt` 條目說明	新增開發語言版本說明與 41 條相依套件表（直接／間接、用途）
4	**完整 Prompt 歷程只在儲存庫、不在 PDF**（只交 PDF 時評分者看不到）	由 `prompt_log/` 自動產生，併入 Chapter 4.4
5	PDF 內網址無法點擊（0 個連結）	連結改為超連結

圖片清晰度檢查：嵌入圖片的有效解析度為 215–251 ppi（列印等級）；但介面截圖中的小字在 13 cm 寬時偏小，改以接近全頁寬度（16 cm）呈現。

結果

- 報告由 32 頁增為 58 頁（A4），章節重整為規範的 4 章加結論與參考資料；Chapter 1 新增 41 條 `requirements.txt` 套件說明表；Chapter 4 內含完整 Prompt 歷程（P1–P28，逐字原始 Prompt）。
- 另存為規定檔名 `C113151143_林奕丞_期中專題.pdf`；PDF 內有 5 個可點擊連結。
- 逐頁檢查：27 張圖片（含 2 張架構圖）有效解析度皆為 178–300 ppi，無內容超出頁面邊界，目錄頁碼與實際頁面一致。

提示工程觀察

- 使用者在最後一刻才附上完整的「繳交規範」圖，使 AI 發現規劃階段（P2）收到的是**不完整的規範**（只有部分內容），導致報告結構與規範不一致。教訓：驗收標準要在開始時完整提供，並在繳交前逐條對照。

5　結論與已知限制

結論：本專案完成四大功能，並以介面層與演算法層分離、完整的輸入防呆、對照 OpenCV 的卷積驗證、PSNR 公式推導與實測分析，滿足作業要求；線上展示與原始碼皆為公開。

已知限制

- Haar-Cascade 以正臉為主，側臉、嚴重遮擋或失焦的臉可能偵測不到；信心度為啟發式正規化，不是機率。
- PSNR 只反映像素差，不反映人眼感知與結構相似度。
- 影像上限 2500 萬像素；超大影像卷積會較慢（約 2000 萬像素、5×5 核約 3 秒）。
- 實測以單一測試影像為主，結論的相對關係具參考性，絕對數值不宜直接類推。

未來工作：加入 SSIM 等結構相似度指標；加入深度學習偵測器（如 MediaPipe）並與 Haar 比較；支援同時檢視多種濾波結果。

參考資料

1. P. Viola and M. Jones, “Rapid Object Detection using a Boosted Cascade of Simple Features,” CVPR, 2001.
2. ITU-R Recommendation BT.601, *Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios*.
3. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, Pearson.
4. OpenCV Documentation：`cv::CascadeClassifier`、`cv::filter2D`。
5. Streamlit Documentation：https://docs.streamlit.io